

Analysis and Vulnerabilities in zkLogin

Sofia Celi

Brave Software

cherenkov@riseup.net

Hamed Haddadi

Brave & Imperial College London

h.haddadi@imperial.ac.uk

Kyle Den Hartog

Brave Software

kdenhartog@brave.com

Abstract

Zero-Knowledge Authorization (ZKA) systems allow users to prove possession of externally issued credentials (e.g., JSON Web Tokens) without revealing the credentials in full via the usage of Zero-Knowledge Proofs (ZKP). They are increasingly promoted as privacy-preserving and decentralized alternatives for authorization, and are already deployed in practice, with proposals for higher-stakes settings such as government access-control frameworks. In this work, we show that the security and privacy of zkLogin—the most widely deployed ZKA system—cannot only be reduced to the underlying ZKP. Instead, zkLogin critically depends on non-cryptographic assumptions about JWT/JSON parsing, issuer trust policy, architectural binding, and execution-environment integrity: none of which are specified or enforced as protocol-level properties.

Via an analysis of the public documentation, source code and surveys on wallets and public endpoints, we identify three broad classes of vulnerabilities in zkLogin: (i) permissive, non-canonical claim extraction that admits malformed JWTs; (ii) transformation of short-lived authentication artifacts into durable authorization credentials without enforcing their issuance context (issuer, audience, subject and temporal validity binding), which enables cross-application impersonation and misuse—particularly in browser-based deployments that expose system’s material; and (iii) systemic centralization and privacy risks arising from reliance on a small set of issuers and outsourced proving infrastructure, including disclosure of user identity attributes to third-party services without consent. We note that none of the vulnerabilities identified are cryptographic in nature. Overall, our findings demonstrate that zkLogin inherits, and in some cases amplifies, fragilities of web-based authentication ecosystems, and that the security of the system cannot be reduced only to the ZKPs.

1 Introduction

Zero-knowledge proofs (ZKPs) are increasingly promoted as a foundational building block for privacy-preserving authentication and authorization systems [3, 13, 15, 25, 33, 36, 39,

60, 74, 76, 82, 90, 97, 106, 113–116, 118, 119]. Recent proposals (particularly targeting integration into blockchain wallets, identity frameworks, and verifiable credential ecosystems) assert that ZKPs enable users to demonstrate ownership of externally issued documents without disclosing the documents themselves. Through this mechanism, such systems claim to enhance users’ accessibility, privacy and security [97]. In these systems, a user receives an external digitally signed document (from an authority), and then produces a ZKP attesting that the document satisfies some authorization predicate and a valid signature. The verifier relies solely on the proof (and public parts of the document), without ever seeing the underlying complete document. We refer to those systems as *Zero-Knowledge Authorization* (ZKA) systems (see Section 2.3).

One example of such a system is zkLogin [8], a widely deployed protocol [4, 95, 101]. zkLogin enables a user to authorize transactions, in the blockchain context, using a proof of possession of a JSON Web Token (JWT) issued through an OpenID Connect (OIDC) login flow. In this architecture, the JWT is treated as a root of trust: the user proves in zero knowledge that the token contains certain claims and carries a valid signature under an accepted identity provider. The verifier, often a smart contract, accepts the proof as an authorization credential, even though it never sees the full JWT. We note that the thread model in zkLogin is one where the backend is untrusted, while the frontend (the app that relies messages) is trusted. At first glance, the security story appears straightforward for this protocol: if the used ZKP algorithm is sound and the issuer’s signature is valid, then authorization of actions should be secure and privacy preserving. However, this narrative implicitly assumes that the credential being proven about is itself well-defined, valid, canonical, and semantically robust. It also assumes that there is a strong link between the issuer, application that received the documents and prover. In practice, none of these assumptions hold automatically.

Here, we show that these issues are not mere curiosities or implementation concerns, but central vulnerabilities of zkLogin’s current architecture and overall system. We identify three broad classes of vulnerabilities in the system. First,

zkLogin implicitly relies on externally issued JSON-based documents that can have ambiguous and non-canonical semantics, despite neither enforcing full JSON validity nor specifying a canonical parsing model. Second, the system transforms short-lived bearer authentication documents into durable authorization credentials, amplifying reliance on issuer trust while weakening standard security properties such as scope binding, replay protection, and temporal validity enforcement. Third, zkLogin introduces privacy and governance risks by recentralizing trust in a small set of issuers and outsourced infrastructure, and by exposing user identity attributes to third-party services outside the original consent relationship. We discuss our findings at Section 5.

This means that, in practice, zkLogin accepts non-canonical JSON-based documents, performs *ad hoc* parsing rules, fails to enforce issuer-binding or scope semantics, relies on long-lived unsafe browser storage for security, and exposes user’s attributes to centralized services. These issues allow malformed documents to yield valid proofs, enable cross-application and cross-subject impersonation, and convert short-lived authentication documents into reusable authorization credentials. Hence, our results demonstrate that the security of this system cannot *only* follow from the ZKPs alone. Instead, it critically depends on external assumptions (document’s correctness, issuer governance, and execution environments) that are not guaranteed by the system.

Scientific relevance. zkLogin is an important case study, both because it is widely deployed in real-world applications and because it represents a new design point in the emerging space of Zero-Knowledge Authorization (ZKA) (see Section 2.3), which have been proposed for use in digital ID systems [2, 22, 38, 100, 111, 112]. Official information from its designers states that there are over 7.6 million zkLogin transactions and over 500k zkLogin addresses [9]¹. From a scientific perspective, zkLogin constitutes one of the first large-scale deployments of a system that derives reusable authorization credentials from conventional web authentication artifacts (OIDC-issued JWTs) via ZKPs. As such, it provides a rare opportunity to evaluate whether the security and privacy benefits promised by ZKA systems in academic settings persist under the full complexity of production identity ecosystems, including heterogeneous issuers, ambiguous JSON/JWT semantics, and deployment-specific trust policy.

The broad hypothesis motivating zkLogin is that ZKPs can reduce reliance on centralized identity providers by minimizing disclosure of authentication data while preserving verifiability. However, prior work on both JWT security and federated identity has repeatedly shown that security tokens are brittle in practice, and that subtle semantic ambiguities often lead to vulnerabilities [37, 62, 63]. Moreover, zkLogin introduces additional challenges by combining web authenti-

cation with decentralized verification and outsourced proving infrastructure. In such settings, trust policy, parsing semantics, and credential’s material storage move outside a single administrative boundary, creating new opportunities for mis-binding and cross-application misuse. One goal of this work is therefore to evaluate whether these attacks can be leveraged in practice, and to characterize the structural mitigations required for zkLogin to remain secure under realistic deployment assumptions.

1.0.1 Our contributions

In this work we provide the following contributions: (i) we reconstruct the overall system design of zkLogin by analyzing their academic paper [8], public documentation [28–32, 46, 54, 98], and code and deployed services [21, 40, 55, 56, 73, 103]; (ii) we identify and validate multiple classes of vulnerabilities arising from non-canonical JWT parsing, missing issuer/audience/subject binding, and insecure environmental assumptions; (iii) we analyze centralization and privacy risks introduced by outsourced proving and token forwarding outside the OIDC consent relationship; and (iv) we propose protocol-level mitigations that treat parsing semantics and trust policy as first-class security properties that should be enforced (and where feasible, proved) rather than delegated to application-specific checks. To the best of our knowledge, the vulnerabilities identified in this work have not been previously documented in the literature. Prior security analyses of zkLogin [80, 120, 121] have largely focused on localized implementation bugs, whereas our results expose broader architectural and ecosystem-level issues that persist even under correct cryptographic implementation. While related types of vulnerabilities have been observed in JWT/OIDC deployments, they have not previously been investigated in the context of ZKA systems, where converting tokens into reusable proofs amplifies impact.

Mitigations. We discuss a set of protocol- and deployment-level mitigations addressing the parsing, binding, and centralization concerns identified in this work. In particular, we argue that issuer trust policy, binding and canonical parsing semantics must be treated as first-class security properties of zkLogin.

Additional findings. We also performed an ecosystem-level survey of some deployed zkLogin integrations, including wallets and publicly accessible proving endpoints [4, 5, 101] (Slush, Surf and Suiet wallets). This helped us highlight practical deployment patterns—such as browser-exposed credentials—that substantially affect the real-world security and privacy guarantees of zkLogin beyond the formal design.

Responsible disclosure and ethics. We disclosed the vulnerabilities described in this work to relevant stakeholders

¹Metrics reported on 28 of March 2025 in the RWC 2025 Symposium.

(see Section 4.1). Our evaluation relied primarily on analysis of public code, documentation, and deployed endpoints, and on minimally invasive surveys designed to avoid impact on real users or service. Where interactions with third-party infrastructure were required to validate findings, we restricted tests to controlled inputs and avoided generating proofs tied to real user accounts. The stakeholders acknowledged the exposure of sensitive system material in the frontend.

Outline. In Section 2, we introduce preliminaries. In Section 3, we provide our reconstruction of the zkLogin system and its underlying design assumptions. In Section 4, we present vulnerabilities arising from non-canonical JWT parsing and missing authorization binding, as well as privacy and centralization risks. We conclude by discussing mitigations and lessons for the design of ZKA systems in Section 5.

2 Preliminaries

In this section, we introduce the preliminaries required for the remainder of the work.

2.1 OpenID Connect

The OpenID Connect (OIDC) Core protocol describes itself as a simple identity layer on top of the OAuth 2.0 protocol [92]. As such, it uses the same parties as OAuth, but with some different names: OAuth 2.0 authentication servers (IdP) are called OpenID Providers (OPs), and client applications (SPs) are Relying Parties (RPs). It also introduces the concept of an end-user, which is a participant using an user-agent (a web browser) for user interactions. As such, the parties of this protocol are:

- **OpenID Providers (OP):** servers capable of authenticating the end-user and providing claims to an RP about said authentication event. After an end-user is registered with such server, they can use an RP with the OP.
- **Relying Parties (RP):** client applications requiring end-user authentication and claims from an OP.
- **End-User:** a user that has an account at the OP, and that aims to authenticate to the RP via its account at the OP-level.

OIDC is a federated-friendly protocol. In practice, an RP cannot send an end-user to authenticate with any OP, unless the RP has been registered there as a trusted application. In OIDC, this registration is necessary so that the RP is provisioned with a `client_id` and has its `redirect_uri` recorded at the OP-level. OPs traditionally make an online portal, web API or the official Connect2id server [16] available for developers to register their RPs. However, how the client registration requests are to be pre-authorised is not defined at

the standard level. Managing access to the registration API is crucial for security, in order to prevent phishing attacks by malicious applications, and an administrative bootstrapping step becomes impractical in large ecosystems involving many OPs and RPs. OpenID Federation 1.0 [88] addresses these limitations by standardising trust establishment and metadata distribution through signed “trust chains”, enabling automated and policy-controlled client registration across the federation. This in turn implies that OPs are not trusted by default: rather, they must be explicitly vetted and admitted into a trust framework, before their documents can be relied upon.

The OIDC flow proceeds as follows: when a user wishes to log in to an RP, they first select an OP from a given list of alternatives. The user is then redirected to the OP, where they authenticate (e.g. via password). The OP then asks the user for consent to log in to the RP and to release some of their data to the RP. The next steps depend on which OIDC flow is used. In the *implicit flow*, the OP sends a signed JWT to the RP via the user’s device. This token includes an issuer identifier identifying the OP (`iss`), a subject identifier identifying the user (`sub`), and an audience identifier denoting the identity (from `client_id`) of the RP (`aud`). This ensures that the JWT will only be accepted by that RP. The JWT may contain additional information, such as the user’s e-mail address, if requested by the RP. In *authorization code flow*, a code is first forwarded via the user’s device to the RP, which is then exchanged for a JWT via a direct channel between the RP and the OP. In this flow, the RP might additionally have to authenticate with the OP.

2.2 JSON Web Tokens (JWTs)

A JSON Web Token (JWT) is a compact, URL-safe encoding of a set of *claims* that are digitally signed (or MACed) to provide authenticity and integrity. A JWT is typically represented as three Base64URL-encoded components separated by dots, `header.payload.signature`, where the header specifies metadata such as the signing algorithm, the payload contains a JSON object of claims, and the signature authenticates the encoded header and payload. JWT claims may be *registered* (`iss` for issuer), *public*, or application-specific private claims. Following the JSON grammar, each claim is divided into “key” and “value”. When authenticated, the signature authenticates the byte-string representation of the token, but the overall security of the JWT depends on the correct interpretation of claim semantics (e.g., enforcing issuer/audience binding, freshness, and intended token use), as well as robust and unambiguous parsing of the underlying JSON (RFC 8259 [10]).

Key Uniqueness. JWT headers and payloads are JSON objects and are therefore subject to the JSON grammar defined in RFC 8259 [10]. While JSON objects are commonly interpreted as mappings from keys to values, RFC 8259 does not mandate how duplicate keys are handled, and explicitly

notes that object member names *should* be unique. In practice, JSON parsers exhibit divergent behavior in the presence of duplicate keys (e.g., rejecting the object, “first wins”, or “last wins”). In fact, RFC 8259 reports that when the keys are not unique, the behavior of software that receives such an object is unpredictable: many implementations report the last key/value pair only; other implementations report an error or fail to parse the object; and some implementations report all of the key/value pairs, including duplicates. This ambiguity is security-relevant in authentication systems: if different components parse the same signed JSON payload under different duplicate-key semantics, then the system may validate one interpretation while using another.

JWTs in OIDC. In OIDC, an RP is expected to perform mandatory validation of a JWT before using it for authentication. In particular, the RP must verify the token’s signature (if present) and validate security-critical claims, including `iss`, `sub`, `aud`, `exp`, and `iat`. This validation is not limited to checking the presence of these claims: the RP must ensure that each claim is well-formed according to the JSON grammar, has the expected type (e.g., correct values for `exp` and `iat`), and satisfies the semantic constraints prescribed by OIDC (e.g., issuer and audience matching the RP’s configuration). Depending on the claim, additional constraints may apply, such as case sensitivity, canonical encoding requirements, and format restrictions.

To validate a JWT, the verifier reconstructs the signed message as the ASCII concatenation of the Base64URL-encoded header and payload, and checks the signature over this string according to the algorithm indicated in the header (typically via the `alg` field) [45]. In OIDC deployments, public verification keys are obtained from issuer-managed key registries (JWK sets), which are typically published at endpoints derived from issuer discovery metadata (e.g., the `jwks_uri` field). The header often includes a key identifier `kid` used to select the correct public key from the issuer’s JWK set. Thus, JWT validity depends not only on signature verification, but also on correct interpretation of header metadata (including `alg` and `kid`) and reliable binding between the issuer claim `iss` and the corresponding public key registry used for verification.

Crucially, OIDC validation is inherently stateful: several checks rely on reference values not contained in the token itself. For example, the RP must verify that `aud` contains the RP’s registered `client_id`, and validate the signature using keys obtained from the issuer’s discovery metadata. OIDC flows additionally require binding the token to the authentication session through values such as `nonce` (to mitigate replay) and `state` (to mitigate CSRF), alongside enforcing application-specific policies. Moreover, these reference values can change over time: signing keys are rotated, client configurations may be updated, and tokens are explicitly short-lived through `exp` and `iat` constraints. These requirements highlight that JWT security is not solely a matter of cryptographic

validity, but of correct semantic enforcement and contextual binding: properties that become particularly delicate in settings where token validation and claim interpretation are decoupled across different components.

2.3 ZK-Authorization services (ZKA)

Over the past years, both government bodies and Internet service platforms have promoted the usage of externally created and managed authentication systems that can be leveraged in order to allow or disallow further access to digital services. In practice, this has been proposed as the deployment of unified identity providers (e.g., national electronic ID systems, federated or centralised login providers) that can signal externally managed authentication via a *ownership-based model* [59]. The appeal of these externally managed systems is typically justified on two fronts [67]. First, *usability*: delegating authentication to widely deployed identity infrastructures reduces friction for end-users, who no longer need to manage multiple credentials (for instance, by the need of memorizing additional passwords or having to backup authentication key material). Second, *policy and compliance*: external identity providers allow authorization decisions to be tied to legally recognized identities or pre-existing account relationships, enabling services to outsource eligibility checks without maintaining their own user databases. In all cases, authorization is performed using signed documents that are created and controlled outside the service’s trust domain.

Recent regulatory proposals, such as the European Union’s Digital Identity Framework (eIDAS 2.0) [23, 24, 51] and similar efforts in the UK [108], US [43, 79, 87], Canada [6], Australia [1], and elsewhere, emphasize the usage of digitally verifiable credentials, particularly those augmented with ZKPs [109], as a foundation for digital authentication that allows for further authorization to services via verification of attributes (e.g., age, residency, nationality). These frameworks envision users receiving cryptographically signed documents from authorized entities, which they can then present to online services to demonstrate eligibility to access further services or perform actions. Similar cryptographic mechanisms are being explored in academic and industry contexts [3, 8, 13, 25, 33, 36, 60, 76, 90, 97, 106, 113–116], particularly in blockchain and decentralised access control systems. While attribute-based verification can enable fine-grained access control, it also inherently introduces risks of censorship and selective service denial. Although we do not focus on these in this work, we refer the reader to [11, 12, 14, 38, 111, 112] for analyses of censorship and harms.

We refer to these systems that use externally issued credentials to authorize further services or actions, via the usage of zero-knowledge proofs for security and privacy, as *Zero-Knowledge Authorization* (ZKA) systems. Examples include *zkLogin* [8], *zkCreds* [90] and more [15, 39, 82]. While these

systems particularly focus on JSON-based identity documents (JWTs and passports represented as JSON documents), the underlying principle is not restricted to neither JSON nor web tokens. However, given the prevalence of JSON-based data on the web [84], it is expected that such documents will most commonly be represented in that format, even if the underlying authorization mechanism does not depend on it.

Formally, we define a ZKA as a system that outsources authorization decisions to cryptographically signed documents issued by external entities, while using ZKPs to keep the documents (or parts of it) private. A ZKA system is a four-tuple system: $ZKA = (\mathbf{IS}, \mathbf{U}, \Pi, \mathbf{V})$, where $\mathbf{IS}$ is a set of signed authentication document issuers, $\mathbf{U}$ is a set of users to whom the document belongs to after performing an authentication action, Π is a prover protocol requested by users to generate a proof that attests the validity of the document, and $\mathbf{V}$ is a set of verifiers (e.g., smart contracts, servers, or distributed systems) of the proof that allow for the authorization of further actions depending on their truthfulness. An issuer $\mathbf{IS}$ generates a signed document as $\mathbf{Doc} = (\text{msg}, \sigma_{\mathbf{IS}})$ where msg encodes attributes relevant to an authorization decision, and $\sigma_{\mathbf{IS}}$ is a digital signature verifiable under $\mathbf{IS}$'s publicly available verification material². The public predicate Φ determines whether the signed document authorizes a protected action: a user is authorized with respect to an action a if $\Phi(\text{msg}) = \text{True}$. A user proves authorization by asking for the generation of a ZKP: $\pi \leftarrow \Pi(\text{msg}, \sigma_I, w)$, where w denotes private witness data (e.g., private attributes from the document and/or the document's signature).

While the security of this scheme formally reduces to the security of the ZKP used [35], the *security of the overall architecture and system* depends on additional assumptions about issuers, credentials, serialization formats, parsing, and verification policies.

3 Overview of zkLogin

zkLogin, introduced in [8], is a prominent instance of a ZKA system, which has been widely adopted across numerous wallets [4, 95, 101] within the Sui [99] ecosystem, and is used to authorize a substantial volume of real-world transactions: public metrics from Dune Analytics show sustained and large-scale use of zkLogin-authenticated transactions (see: <https://dune.com/queries/6273575>). Here, we provide a full view of the zkLogin ecosystem and components. To do this, we carefully analyze the full system and conduct an analysis of the current live integration: our analysis draws upon the zkLogin academic paper [8], open-source software [21, 40, 55, 56, 73, 103], surveys conducted on the official

live integration³, official demo [46, 98], public documentation [28–32, 54], and public security audits [80, 120, 121]. We emphasize that our analysis proceeds on two fronts: (i) the reference implementation maintained by the platform (which is partly open-source), and (ii) the official documentation that enables external developers to deploy their own zkLogin service. This is partly due to the fact that zkLogin can be implemented by any developer based on public documentation, while several widely used official deployments and components are not fully open-source. In the following, we present the architecture and workflow of zkLogin.

3.1 zkLogin's Design.

zkLogin enables users to authorize *actions* (e.g., transactions) using a signed document issued by an external OIDC Identity Provider (OP) [27] (in this case, the $\mathbf{IS}$, see section 2). The core idea is that, instead of maintaining independent application key material, a user proves in ZKP that they possess a valid JWT (the document) signed by an issuer (e.g., Google, Twitch, Facebook), and that a prescribed subset of the JWT's claims satisfies a specified predicate. zkLogin introduces six entities for this flow: an external OP, which acts as the $\mathbf{IS}$; an external proving service acting as Π ; the RP, which is the client (a website or an app) acting on behalf of the user; a user $\mathbf{U}$ which uses the RP; an external salt service; and a verifier (acting as $\mathbf{V}$) that verifies the proof based on public data.

The architecture is composed of five main components (listed here in no specific order): (i) *registration*, whereby parties register identifiers with each other; (ii) *request*, whereby an RP requests user authentication via OP before executing certain actions and receives a signed JWT on success, (iii) *issuance*, whereby an external $\mathbf{IS}$ issues signed JWTs via an OIDC authentication flow; (iv) *proving*, whereby a proving service, deployed externally⁴ constructs a ZKP proving claim inclusions on the JWT and some cryptographic computations, and verifies that the JWT's signature is valid under $\mathbf{IS}$'s public key (by maintaining a local mapping of $\mathbf{IS}$ names and their public keys); (v) *salt generation*, whereby a salt service, deployed externally, provides with "salt" material; and (vi) *verification*, whereby an action verifier, typically a smart contract, verifies the proof using public verification keys and material, and authorizes any corresponding user actions.

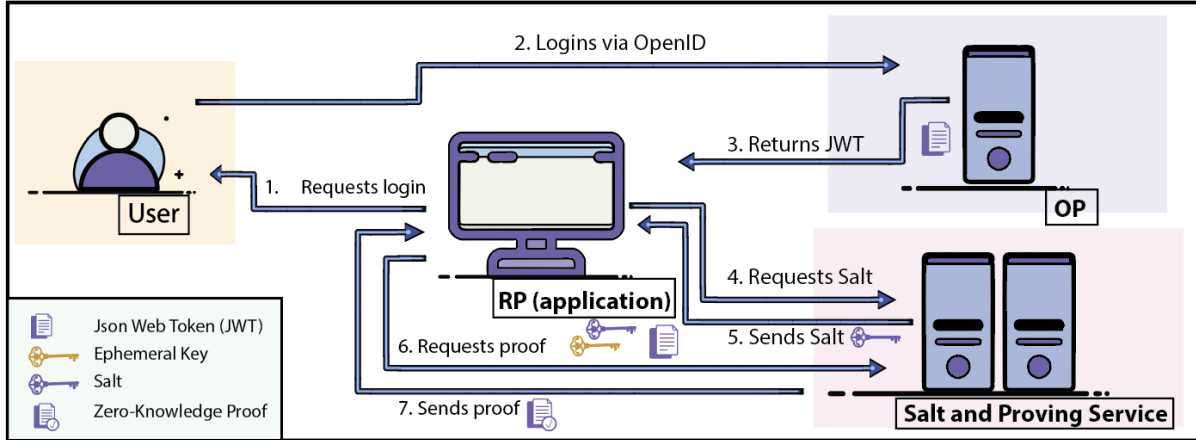
In the following, we first briefly discuss zkLogin's threat model and then introduce the different flows of zkLogin (see Figure 1): registration, request and issuance of JWT, salt-generation, proof-generation and verification. We note that throughout, when we say "session" we refer to an *OIDC authenticated session* as instantiated for a given login at the

²This verification material may consist of a public key, a certificate chain, a JWKS descriptor, or any other authenticated artifact that allows verifiers to check signatures issued by $\mathbf{IS}$.

³We note here that the proving and salt server's code is not open source; we reconstruct its behaviour from public documentation and analysis of wallet integrations.

⁴While proof generation can be executed client-side, current guidance and implementations of zkLogin recommend outsourcing it due to the high computational costs involved.

Figure 1: A simplified overview of zkLogin: the system is illustrated up to the generation of the proof.



RP: namely, the period during which the RP treats the user as authenticated based on a particular OIDC-issued JWT or their own configuration. As so, this session is effectively bounded by the JWT’s validity window (e.g., until `exp` expires), by the RP retaining the JWT (e.g., in memory or local storage), and by avoiding replay via a `nonce` claim. However, zkLogin effectively establishes authorization sessions that may outlive individual OIDC sessions. In particular, a zkLogin “session” is instead tied to zkLogin-specific material that may be retained by the RP. Consequently, a zkLogin session can remain usable as long as this material persists, thereby enabling authorization to continue across multiple OIDC sessions and weakening the intended short-lived semantics of JWT-based login. We refer to this zkLogin sessions as “epochs”.

3.1.1 Threat model

zkLogin, as noted in the paper [8], assumes that the “backend” services are untrusted, except for the **IS** which is assumed trusted. This is somewhat in line with trust assumptions in the OIDC ecosystem, where OPs are not trusted by default but rather *only* when an explicit trust relationship is established (federation trust chains). While OIDC mandates validation of JWTs (signature verification and checks on JWT’s claims), these checks only establish that the JWT is valid with respect to the **IS** [72]. Nevertheless, OIDC is a federated-friendly protocol so parties can deploy an issuer endpoint (via common identity platforms or self-hosting) and produce signed JWTs. In particular, the OIDC Core protocol provides no global mechanism to detect or exclude malicious issuers: trust in issuers is an application-level decision, unless complemented with federation mechanisms such as OpenID Federation trust chains. Moreover, even when issuers are not adversarial, real-world OIDC deployments frequently deviate from the idealized protocol model [64, 70, 93]. Issuers are often misconfigured or implement incomplete subsets of the OIDC

specification, resulting in JWTs whose claims do not robustly enforce the intended authentication context [50, 77, 102, 104]: missing or weak enforcement of replay protections (absent `nonce` checks), inconsistent use of audience-related claims, overly permissive JWT lifetimes, and unexpected claim formats or optional fields. Unit 42 [77], for example, reports critical OIDC misconfiguration patterns in CI/CD ecosystems affecting major vendors such as CircleCI and GitHub Actions. Such deviations can be survivable in conventional web authentication because the RP can compensate with additional checks and session binding. Thus, unless the RP, prover or salt service enforce a strict issuer allow-list policy (and root so in a trust chain), a malicious or attacker-controlled issuer can generate JWTs that pass validation while violating the RP’s intended authorization policy.

Further, in zkLogin, the trust boundary differs from that of standard OIDC deployments. Concretely, the RP is treated as trusted and not needed for security. In practice, hence, the RP cannot be assumed to reliably enforce OIDC-mandated validation steps in the presence of a malicious or compromised backend, since such checks are neither specified nor required by zkLogin itself. As a result, in the model, an RP may omit OIDC validation entirely and forward malformed JWTs to the proving and salt services, as long as it forwards the document and retains required material in memory.

Under this model, the proving and salt services occupy an ambiguous trust position. Depending on the integration, they may be operated by the RP itself or by external infrastructure providers. In the latter case, the security model becomes especially brittle: a third-party proving service typically has no direct relationship with **IS** and limited context about the RP’s registration and policy constraints. As a result, these services may accept documents largely at face value, effectively shifting responsibility for correct JWT validation away from the RP without providing any issuer-authoritative enforcement mechanism. This trust ambiguity is particularly problematic

because the verifier cannot observe the underlying JWT contents and therefore cannot independently apply conventional OIDC validation logic. Consequently, issuer- and deployment-specific inconsistencies that would otherwise be detectable and containable become a source of systematic fragility, undermining the security assumptions of the authorization layer. As we will see, this mismatch directly enables several of the vulnerabilities identified in this work.

We note that other than stating which services are trusted or untrusted, zkLogin provides no concrete threat model. In particular, the proving service is not explicitly discussed in their adversarial analysis, despite playing a central role in the protocol. In this work, we therefore consider three threat models (as in prior works [83]) by focusing on the three sets of parties involved in zkLogin: (i) the RP, the application running on the user’s device; (ii) external services (salt and proving servers); and (iii) the OP issuing JWTs. Concretely, we consider the following adversaries: (1) a network adversary that can monitor and tamper with communications between the RP and the external services and OP; (2) an adversary that has compromised the external services (prover and/or salt service), and/or the OP infrastructure; and (3) an adversary that compromises the device running the RP for a short period of time (e.g., via a malicious browser extension).

The first model corresponds to the standard adversary for network protocols, often called the Dolev–Yao adversary [20]; it will be referred to as *standard model*. The second model is one that any reasonable security analysis must consider, since external servers (salt, prover or OP) can be maliciously controlled: hacking groups have consistently targeted blockchain applications [17, 48] in order to gain persistent access to services; this model will be referred to as *malicious-services model*. The third model is particularly relevant for zkLogin deployments, since the RP persists long-lived authorization material (e.g., salts, API keys, and epoch-related cryptographic values) in browser-accessible storage or memory; this model will be referred to as *compromised-RP model*.

3.2 zkLogin Flow

Registration. Before an application can use OIDC, it must first register with the OP (the **IS** or identity provider) [27, 29]. This registration results in the **IS** assigning the RP a public identifier `client_id`, which is subsequently used by the **IS** to recognize it and to populate the audience claim (`aud`) of issued JWTs (note that `aud` may be a list). Importantly, `client_id` is not a private immutable or unique cryptographic identifier: it is a public administrative identifier whose validity and lifetime are controlled by the **IS** (e.g., it may be rotated or revoked). As a result, `client_id` provides only a *policy-level* binding between the **IS** and RP, rather than a stable cryptographic authorization binding. After this registration, the RP additionally directly registers its `client_id` with the proving and, optionally, salt services. This procedure introduces

an extra trust dependency: the external services rely on the supply of the correct `client_id`, and are assumed to consistently enforce the intended OIDC registration semantics, even though backend services are untrusted. Finally, external services maintain an issuer allow-list and corresponding mappings from each trusted issuer to its public key registry (e.g., JWK endpoints), which are used to validate JWT’ signatures.

Login and issuance of JWT. The workflow begins with an **U** attempting to log in to an application (most commonly, a wallet) that delegates authentication to an external **IS** [29].

- an ephemeral key pair (vk_U, sk_U) , stored on the user’s device, which will be used to authorize actions: these are values persisted as part of epoch material;
- a random value r , persisted as part of epoch material;
- a maximum expiry bound T_{max} , which determines the intended validity period of the epoch material (and may be longer than the lifetime of the JWT and session);
- a nonce (persisted as epoch material) computed as $nonce \leftarrow H(vk_U || T_{max} || r)$, which is subsequently embedded into the JWT via the OIDC `nonce` claim.

The expiry bound T_{max} must be expressed in a unit meaningful to the verifier (e.g., in blockchain deployments, an epoch number). To prevent applications from selecting arbitrarily long-lived keys, the verifier may enforce an upper bound such as $T_{max} < T_{cur} + \delta$, where T_{cur} denotes the current epoch and δ is a system-defined maximum window. The inclusion of r in the nonce is intended to provide unlinkability across epochs: it prevents the **IS** from learning vk_U during authentication (since `nonce` hides vk_U), and thus hinders **IS** from linking subsequent actions to a stable public key. Note that `nonce` can also embed the epoch’s expiration time.

Once those values are generated, a user authenticates through a standard OIDC login flow. As part of this process, the RP embeds the previously computed `nonce` into the OIDC authorization request sent to the **IS**. In OIDC, the `nonce` parameter is a standard request field included by the client in the authentication request, and **IS** is required to copy it verbatim into the JWT [91]⁵. After a successful login, the **IS** returns a signed JWT containing required and non-required claims such as the issuer (`iss`), subject (`sub`), audience (`aud`), nonce (`nonce`), expiration (`exp`), and many more, along with the digital signature over the document (the presence of the signature, means that the JWT must have a header that conforms with it [44]). We note here, though, that this design decouples the lifetime of the zkLogin authorization material from the OIDC session, as the tuple (vk_U, sk_U, r, T_{max}) (and hence `nonce`) remain usable beyond the validity of the issued JWT (as stated in the `exp` claim). In other words, zkLogin establishes an

⁵Note though that the **IS** can ignore this request.

authorization epoch whose duration may exceed that of the underlying OIDC session. Furthermore, zkLogin reinterprets the meaning of the `nonce` claim: in standard OIDC [91], the `nonce` serves as a well-defined replay-prevention role per session: must be unpredictable per session, and the RP must verify that the value in the JWT matches the locally generated nonce. In zkLogin, by contrast, `nonce` is repurposed primarily as an unlinkability mechanism per epoch and is not validated against stored session state, undermining its replay-prevention purpose. Moreover, documentation indicates that the nonce (or parts of it) may be stored in browser storage [110] and reused across logins [46].

As noted, the **IS** returns a JWT that contains some claims. In practice, zkLogin expects a small number of claims to appear in the JWT, and ignores all the rest. Per documentation, the JWT’s payload, in a mandatory manner, requires four claims: the `iss`, `aud`, `sub` (with the additional requirement that when `sub` is an email address, `email_verified` must be set to true⁶), and `nonce` supplied by the RP. All additional claims such as `exp`, `iat` (issued-at), `azp` (authorized party), or `jti` (JWT identifier) are treated as optional, and are neither required nor validated. This means that conformance checks of JWT’s claims appear to rely almost exclusively on the presence of some of claims and of a valid signature, and do not align with the requirements defined in the OIDC Core specification [91] (see section 2.2) or contradict them. Likewise, the JWT header is only partially interpreted: zkLogin only checks that the `alg` field is literally the string “RS256”⁷ and reads the `kid` field. The system will use `kid` to select a key from a external public mapping of issuers to their advertised public signing keys. Notably, because each services may reuse JWTs and auxiliary material, the mapping may include keys that are no longer active at the **IS** side.

Preparing for proof generation. Once the **IS** issues a JWT token, it is returned to the RP, rather than being sent directly to the proving or salt services. At this point, zkLogin admits two deployment patterns. First, the application may store the JWT on the user’s device and later reuse it when requesting a proof. Second, the application may avoid persistence and directly forward the received JWT (or relevant parts of the **IS** response) to the proving service. In the persistent case, zkLogin provides no explicit guidance on secure document retention, isolation boundaries, or JWT lifetime management, despite the JWT constituting a bearer credential in the threat model. In the non-persistent case, forwarding the JWT immediately reduces local storage exposure but still relies on correct semantic validation prior to proof generation.

According to OIDC [91], the RP is expected to validate

⁶Notably, this additional `email_verified` constraint is mentioned in the paper but not enforced by the documentation or publicly available implementations.

⁷The inclusion of only this algorithm seem to be done in order to prevent `alg == none` attacks.

the JWT prior to use, including verifying the signature, at-testing to well-formedness, checking the issuer and audience claims, and enforcing temporal validity (e.g., via `exp` and, where applicable, `iat`.) However, the zkLogin paper recommends *only* signature verification at this stage and provides no guidance on enforcing expiration or other mandatory semantic checks. Consequently, any JWT that verifies under the issuer’s key—including JWTs that are expired or carry inconsistent claim semantics—may still be forwarded for proof generation. Further, the design implicitly assumes that the user will perform a fresh authentication whenever needed and that repeated logins yield equivalent JWTs (or at least stable values for (`iss`, `aud`, `sub`, `nonce`)). In practice, these values are neither guaranteed to be stable nor sufficient to capture the intended freshness and scoping semantics of the OIDC model. As a result, both persistent and non-persistent workflows effectively treat JWTs as semantically static inputs, weakening their intended role as short-lived, context-bound authentication artifacts.

Since subject identifiers (`sub` claim) may be stable across applications for many **IS** (when public subject identifiers are used), the zkLogin design introduces a `salt` to prevent direct linkage between this user’s OpenID identity and their user’s blockchain address. Without a salt, users logging in would repeatedly derive the same blockchain address, making their activity trivially linkable. To mitigate this, zkLogin proposes deriving the on-chain address as `zkaddress = H(sub||aud||iss||salt)`, where the salt is a user specific random value: it can, however, live longer than sessions. For this, the RP must either retrieve or generate locally this `salt`. When the first happens, the application forwards the JWT (or parts of it) to an external salt service: no strict checks, besides signature verification, are enforced at this level on the JWT. This design introduces an additional privacy risk: transmitting the JWT in the clear to a third-party service reveals identity attributes beyond what the user has consented to disclose. The zkLogin paper suggests mitigating this exposure using privacy-preserving mechanisms such as MPC or TEEs; however, public documentation and widely used deployments provide limited evidence that these alternatives are used in practice. The returned salt must be stored client-side (in the reference demo and paper, it is simply placed in browser local storage), and the zkLogin documentation provides no guidance regarding secure storage practices or lifetime management: in fact, it argues that persisting the salt on local storage is reasonable since it is less sensitive than a password. Further, the salt can be reused indefinitely as long as it remains present in local storage.

zkLogin emphasizes that the salt is recommended for privacy, but not universally desirable. If **IS** already issues pairwise (per-client) subject identifiers, then the `sub` may already provide unlinkability, reducing the need for a salt, and, when users prefer discoverability, it may be omitted. Importantly, the design explicitly acknowledges that the use of salt cannot

be enforced: the untrusted service or RP may set the salt to a public value (including zero), effectively disabling unlinkability. Note, though, that losing the salt permanently renders the derived `zkaddress` unusable, as well as any action associated with them, and leaking it allows any other party possessing the relevant JWT’s claims to derive the same address. Moreover, note that the salt is not cryptographically bound to either the specific RP or the user’s device that generated it. It is simply treated as an opaque, persistent value: any application that learns the `salt` can reproduce the same `zkaddress`, and any device that stores it can later reveal it. Thus, privacy depends solely on the secrecy and permanence of a RP-managed potentially externally-generated random value.

Proof generation. Once the RP has obtained both the JWT and the corresponding `salt`, it requests a zero-knowledge proof. In zkLogin, proof generation is typically *outsourced*: instead of producing the proof locally, the client forwards the JWT, `salt`, $(vk_U, T_{\max}, r)$, and `zkaddress` to an externally operated proving service. In the official demo and documentation, the JWT is expected to be transmitted to the proving service as a regular JSON field in an HTTPS POST request, rather than via browser-protected credential channels (e.g., cookies bound to an origin). As a result, standard browser credential protections do not apply to the JWT as it is handled as an explicit application payload, increasing exposure to exfiltration and replay in the presence of a compromised client environment.

Upon receiving the request, the proving service performs a minimal origin check by verifying that the JWT’s audience claim `aud` matches a registered `client_id`⁸. However, the design provides limited guidance on how audience values should be registered, how they map to RPs identifiers, or how to ensure that this configuration is issuer-authoritative.

After this minimal audience check, the proving service briefly validates that the JWT parses correctly (via the procedure below), and then executes the proof-generation circuit. For this, the proving service will need to attest that: i. the JWT is well-formed via a “*ad-hoc* selective parsing”, and ii. that its signature is valid. For the former, rather than implementing a full JSON parser, the circuit extracts only a small subset of the JWT payload claims, and does not enforce JSON validity or canonicalization. Instead, it performs a sequence of syntactic substring searches to locate specific top-level key-value pairs corresponding to the required claims. We refer to this approach as *ad-hoc selective parsing*. The zkLogin paper describes this extraction procedure (applied only to a single-instance of the four required claims) as follows, for an input string S (representing the JWT payload):

1. Given a starting index i and a length ℓ , extract a substring $S' = S[i : i + \ell]$ from the input string S .
2. Check that the final character of S' is either a comma ‘,’ or a closing brace ‘}’.
3. Given an index j corresponding to the colon, check that $S'[j]$ is the character ‘:’.
4. Interpret $S'[0 : j]$ as the key (`key = "sub "`) and $S'[j + 1 : -1]$ as the value (`value = "user"`).
5. Verify that both key and value of claims are JSON strings by checking only that their first and last characters are quotation marks, tolerating some whitespace variations via similar substring checks. Note that this does not enforce the JSON grammar: characters inside the string are not validated, escape sequences are not checked, and control characters or embedded quotation marks may appear unescaped. This is non-conformant with JSON string-encoding rules as mandated by RFC 8259 [10].

These operations are repeated only for one instance of the fixed, hard-coded set of required claims (`iss`, `aud`, `sub`, and `nonce`), and are not performed at the JWT’s header level, as the header is treated as public data. The circuit does not enforce unique keys, canonical ordering, correct JSON nesting, or well-typed values, nor does it ensure syntactic integrity outside the extracted substring windows. In effect, the proving system does not validate that the JWT payload is a well-formed JSON object. Instead, it checks only that certain byte slices *resemble* quoted strings terminated by either ‘,’ or ‘}’. Because claims are located solely via positional substring logic, the circuit provides no guarantee that a claim is unique or unambiguous.

With these assumptions, the proving circuit (and its associated proof π) ultimately attests to four properties: (i) that the above “*ad-hoc* selective parsing” procedure correctly extracts the expected top-level fields from the provided byte string according to this circuit’s own *ad-hoc* parsing rules; (ii) that the JWT’s signature verifies under the public key corresponding to `IS` and the algorithm specified in the header; (iii) that the `nonce` value was computed as $H(vk_U || T_{\max} || r)$ for some vk_U ; and (iv) that the claimed user’s address `zkaddress` was correctly derived from the extracted claims and the salt. Note that neither the proving service nor the verifier stores the `nonce` or `salt`⁹ or maintains any state about previously used values, and it doesn’t contact the blockchain to check whether the expiry parameter $T_{\max}$ is still valid. As a result, no component in the zkLogin architecture is able to detect or prevent reuse of a JWT whose `nonce` or `salt` has been consumed and expired.

⁸The `aud` value is chosen by the `IS` based on `client_id`, yet the proving service does not directly participate in the issuer registration process; consequently, it must rely on RP-provided configuration when determining acceptable audience values.

⁹It appears, though, that in some deployments, the `salt` is stored, but this is not clarified in the documentation.

Verification and executing actions. The RP receives π and the JWT’s header, which validity is only linked to the nonce’s validity and can stay in memory even after the user’s logout. When the user is ready to execute an action, the application signs the action payload tx using the ephemeral private key sk_U generated earlier, and generates the signature σ_U . The full zkLogin authentication artifact attached to an action is then $(vk_U, T_{\max}, \sigma_U, \pi, \text{JWT’s header}, iss)$. A verifier receiving this transaction then:

1. Checks that pk_{IS} is a current public key of the issuer specified by iss . This requires an oracle procedure that periodically fetches iss ’s JWK set from its public endpoint and posts any accepted keys on-chain. Keys seen within the last Δ epochs are assumed to be valid. Crucially, per the official paper, this mechanism does not enforce that the specific signing key used in the JWT corresponds to any particular key identifier (e.g., kid) provided in the JWT; it merely asserts that the signature verifies under *some* observed key of the issuer. However, the documentation and code do seem to perform the check against the header’s kid .
2. Checks that the expiration parameter satisfies $T_{\max} \geq T_{\text{cur}}$ and $T_{\max} < T_{\text{cur}} + \delta$. Notably see here that there is no check that $T_{\max}$ corresponds to the JWT nonce’s $T_{\max}$.
3. Verifies π using the public $P = (pk_{IS}, iss, zkaddr, T_{\max}, vk_U, \text{JWT’s header}, iss)$. This attests to the correctness of the “*ad-hoc* selective parsing”, to the JWT’s signature under the indicated issuer key, the correctness of the `nonce` computation and the derivation of `zkaddress`, and verifies the action signature using vk_U .

By doing these steps, zkLogin transforms a short-lived authentication JWT into a reusable authorization credential (used by various external services) mediated purely through ZKPs, with the issuer’s signature functioning as a cryptographic root of trust for actions, despite the issuer never interacting with the action-enforcer directly. Although the JWT remains signed, zkLogin no longer treats it as temporary, but as a long-term authorization artifact whose validity does not depend on expiration, session state, or audience enforcement.

zkLogin Demo and Documentation. The Sui Foundation documentation links to a community-maintained zkLogin example and walkthrough [31, 46] that explicitly illustrates how developers are expected to integrate zkLogin. The example decomposes zkLogin into seven steps (i. generate ephemeral key pair, ii. fetch JWT, iii. decode JWT, iv. generate the `salt`, v. derive Sui address, vi. fetch ZKP, vii. assemble zkLogin signature) and runs entirely in the browser, using a Mysten Labs maintained proving service as backend [31]. All sensitive inputs required for proof generation

are stored in the browser’s storage: (vk_U, sk_U) and r are kept in `sessionStorage`, whereas the `salt` and maximum epoch are stored in `localStorage` [31, 47]. The documentation explicitly notes that, as long as the browser’s `localStorage` is not cleared, a user can reuse the same JWT (e.g., by logging in with the same **IS** account) to access the corresponding address at any future time [31]. This directly demonstrates that JWTs are treated as reusable inputs for ZKPs, without any enforced notion of freshness, nonce replay protection, or JWT expiration on the client or proving-service side.

From an architectural perspective, the demo confirms the design pattern we analyzed above: (i) the JWT is delivered to RP, not to the proving service; (ii) the client forwards the JWT (or selected fields) to an externally managed proving service operated by a third party; (iii) the proving service is invoked over a generic HTTPS endpoint and has no direct channel to the **IS** to check audience binding, revocation status, or expiration; and (iv) salts are managed entirely on the client, with long-term storage in browser local state, rather than being cryptographically bound or stored. The official documentation further emphasizes that no additional backend is required beyond using the hosted proving service, thereby cementing a deployment model in which a single centralized prover receives JWTs (or their decoded contents) from many independent relying parties and users, and in which the lifetime of a JWT is effectively governed by browser storage and chosen $T_{\max}$, not by its `exp` or OIDC semantics.

4 zkLogin Vulnerabilities

In what follows, we report vulnerabilities in the design, documentation, and implementation ecosystem of zkLogin. Our methodology mirrors Section 3: we analyze the zkLogin academic paper [8], relevant open-source components [21, 40, 55, 56, 73, 103], the official demo [46, 98], public documentation and integration guides [28–32, 54], and public security audits [80, 120, 121]. We additionally survey multiple wallets that integrate zkLogin in practice [4, 5, 101] and survey official proving endpoints¹⁰.

The vulnerabilities we report do not rely on breaking cryptographic primitives or the soundness of the underlying ZKP scheme. Instead, they arise from inconsistencies in JWT/OIDC validation, *ad-hoc* claim extraction, missing issuer–audience–RP binding, brittle environmental assumptions, and issuer trust boundaries. Each vulnerability either (i) yields a valid zkLogin proof from a JWT that would be rejected under standards-compliant OIDC validation, or (ii) enables reusable authorization proofs that are not correctly bound to the intended RP or subject identity. Where applicable, we validated our findings against the open-source zkLogin implementation [55], third-party proving infrastructure [21],

¹⁰<https://prover-dev.mystenlabs.com/v1> and <https://prover.mystenlabs.com/v1>

and the official integration guidance [30, 32].

More broadly, we argue that zkLogin should not merely provide a mechanism for producing ZKPs about JWT contents, but should *standardize and authenticate the semantics* of what those proofs mean. In particular, security-critical policy decisions—such as which issuers are trusted (an issuer allow-list) and how JWTs are parsed and canonicalized—must be treated as part of the protocol definition and verification statement, rather than as local deployment choices. Otherwise, the system devolves into “proofs of strings” whose interpretation depends on undocumented parsing conventions and mutable out-of-band configuration, enabling cross-deployment inconsistencies and claim-confusion exploits.

Threat model. We focus on a conservative setting in which the proving and salt services are *untrusted* but follow their stated interfaces (i.e., they are not assumed to actively help an attacker). Equivalently, we treat these services as semi-honest: they may deviate in terms of policy enforcement or validation strictness, but they do not deliberately collude with an attacker to craft malformed proofs. This assumption is *weaker* than the adversarial model discussed in the zkLogin paper, which treats backend services as malicious. Importantly, the vulnerabilities we describe do not require cooperation from the proving or salt services; rather, they arise from missing or underspecified validation and binding checks that are absent by design. Therefore, our attacks apply *a fortiori* in settings where these services are fully malicious.

For the remaining parties, we adopt stronger adversarial assumptions. We consider the OP in the *malicious-services-model*: the OP may be attacker-controlled, or simply misconfigured, and may issue JWTs that are signed yet violate the intended semantics. We further consider the RP in the *compromised-RP-model*: the RP may be compromised, or affected by runtime client attacks (e.g., injected scripts, malicious browser extensions, compromised dependencies, or brief device access). In particular, a compromised RP may forward malformed or semantically invalid JWTs to external services, and persist long-lived authorization material in browser-accessible storage or memory. We note that this assumption directly contradicts a key trust claim made by zkLogin. In particular, zkLogin explicitly assumes that the application’s front-end is trusted while the back-end is untrusted, arguing that front-end code is public and thus subject to public scrutiny [8]. Moreover, zkLogin claims that the application is required for liveness but not for security, i.e., that even a malicious app cannot steal the user’s assets. However, in the browser-based deployment model encouraged by zkLogin, the trustworthiness of the RP front-end cannot be reduced to source-code visibility: injected scripts, malicious extensions, compromised dependencies, or brief device access can obtain runtime access to sensitive material stored in browser-accessible memory or storage, enabling extraction and replay of long-lived authorization artifacts. This has been widely

explored in research [34, 75, 78, 117].

We argue that our overall setting is realistic: OPs exist outside the administrative boundary of the system and can be misconfigured or adversarial; zkLogin encourages outsourced proving and salt generation; and many deployments are browser-based, where the RP environment provides limited isolation guarantees. Accordingly, zkLogin should remain secure even when deployed with third-party proving and salt services and across heterogeneous issuers and RPs that do not share a single administrative trust domain.

4.0.1 Semantic confusion vulnerabilities in zkLogin’s JWTs

In this subsection, we consider different types of vulnerabilities that are introduced with malformed JWTs. Because JWTs are issued and signed by an OP, malformed JWTs can arise either intentionally (malicious **IS**), accidentally (non-compliant), or through compromised RP side processing steps that reconstruct, wrap, or otherwise transform token contents prior to proof generation. This is not a constrained scenario, as research shows that malformed JWTs can be issued by malicious or non-malicious **IS**, or malformed by RPs [26, 65, 71], specially when working in a federated-friendly system as OIDC. Because zkLogin does not enforce JWT’s claim validation and integrity checks by the RP, validation is implicitly delegated to the proving service, who becomes the *first and only entity* to partially validate the JWT prior to its use as a long-term authorization document. However, we note that, as highlighted in the previous section, the procedure used to validate JWTs at the external services side is not compliant with either the JWT specification [45], the JSON grammar (RFC 8259 [10]), or OIDC requirements [91]. More importantly, it does not enforce even minimal syntactic or semantic security properties expected of security tokens.

zkLogin thus marks a wide variety of malformed JWTs as valid, by only using the “*ad-hoc* selective parsing procedure”. Concretely, our analysis of the official proving endpoints and reference implementation show that the prover-side parsing logic accepts a broad class of syntactically invalid or semantically ambiguous JWTs. An example illustrating several of these issues appears in Figure 2. We note the most security-relevant classes of malformed-JWT that zkLogin accepts:

1. **No semantic header validation (only a size bound).** The JWT header is treated as a public information with the justification that no sensitive information is present on it. Note though, while the header does not typically contain confidential information, it *does* contain security-critical metadata, including `alg` and `kid` used for signature verification: we will note this assumption later as a privacy leak. With this “public header” assumption in mind, the service only attests that the JWT header satisfies a fixed length bound, without validating its structure or semantics. This appears to be an *ad-hoc* mitigation intended to prevent inclusion of fields such

as `jwt` (which could enable self-issued JWTs). However, it provides no meaningful integrity guarantees: malformed or semantically inconsistent headers under the bound are accepted, while standards-compliant headers (e.g., long `kid` values) are rejected. This breaks interoperability without preventing header-based exploits in a principled way.

2. Non-canonical claim extraction enables claim confusion (duplicate keys, shadowing, and character smuggling). The proving service does not enforce that JWT payload keys are unique, nor that a single canonical JSON parsing procedure is used consistently across components. As a result, an attacker can craft payloads containing multiple occurrences of the same claim key, e.g., `{"iss": "honest", "iss": "malicious"}`. Since JSON implementations often disagree on duplicate-key handling (“first wins”, “last wins”, or reject), this creates parser differentials between (i) prover-side claim validation and (ii) the claim values bound into the proof statement.

In our measurements, the prover’s *ad-hoc* selective parsing effectively implements *shadowing* semantics by selecting a particular occurrence (e.g., last-wins), allowing an attacker to override previously validated values by appending a malicious value somewhere else in the payload. More generally, the proving service accepts non-canonical key encodings (e.g., escape sequences, Unicode escapes, or prefix characters) without normalization or rejection. If any component performs *ad-hoc* string matching on the raw payload rather than JSON-decoded keys, then syntactic variants may be interpreted inconsistently across the stack (prover, circuit/statement generation, and verifier/RP), amplifying claim-confusion exploits.

3. Issuer checks are prover-local via static allow-list. The `iss` claim is validated *only* by checking membership of its value in a fixed hard-coded list of providers chosen by the prover or by the external services. At protocol level, neither the verifier nor RP validates this allow-list¹¹, or they can maintain a different allow-list, nor is the allow-list authenticated/bound into the proof. Thus, issuer acceptance becomes prover-defined policy (or external-services defined policy) that may drift over time as the list is updated, and may diverge from the RP’s intended issuer policy. This is more serious given that the prover service is considered untrusted.

4. Non-compliant handling of `aud`. Although JWT allows `aud` to be either a string or an array, zkLogin rejects JWTs with array audiences. This breaks compatibility with standards-compliant issuers and encourages issuer-specific workarounds, increasing the risk of inconsistent audience handling across deployments.

¹¹We note that in the zkLogin documentation there is no encouragement that RP should validate this allow-list and we also note that RPs can deploy their own prover if so they want.

```
{
  // ----- Header -----
  "alg": "none", // bogus algorithm
  "alg ": "RS256", // trailing space
  "alg": "RS256",
  "kid": "992475",
  "typ": "JWT",

  // ----- Payload -----
  "iss": "https://accounts.example/", // honest OP
  "\iss": "https://evil/", // injected
  "iss ": "https://evil2/", // trailing space
  "\u0069ss": "https://ex/", // unicode
  "sub": "C37900",
  "sub ": "MALLORY", // duplicate
  "sub ": "<script>alert('hi')</script>",
  "aud ": ["5731200", "other.."], // not allowed
  "nonce": "gIFt7xtjGLZq5cC0-TgIEeIcuJM",
  "nonce ": "AAAAAA", // last-wins
  "exp": 1716265381,
  "nbf": "not-a-number" // wrong type
}
```

Figure 2: Example of a malformed JWT accepted by zkLogin’s proving services. Despite violating JSON grammar, duplicating critical claims, and shadowing claims, the JWT is still parsed as valid.

5. Temporal claim validation is missing. Temporal fields such as `exp` and `iat` (mandatory in OIDC) are neither required nor checked. As a result, expired or future-dated JWTs are accepted as long as they carry a valid signature. This enables replay of stale JWTs (as the `nonce` claim is not used as a replay protection but rather as an unlinkability measure) and weakens session-bounding guarantees.

Said issues compose into practical vulnerabilities: ambiguous claim parsing enables claim shadowing, which in turn allows the prover to validate benign values while binding malicious values into the proof. Together, these deviations effectively define a *non-standard JWT grammar* and enable claim-confusion exploits. While the JWT signature prevents an adversary from altering claims without issuer cooperation (or signature forgery), zkLogin’s permissive and non-canonical parsing causes different components to interpret the same payload according to different claim-extraction semantics. Moreover, accepted claim values may contain attacker-controlled substrings, including HTML or JavaScript fragments (e.g., `<script>...`). In practice, JWT claims are often routinely logged, forwarded, or embedded into service infrastructure and wallet UI code. Thus, accepting non-canonical or unsanitized claim strings expands the attack surface for *indirect exploits* (e.g., injection into logs), consistent with prior work on JWT misuse and claim-injection vulnerabilities [62, 94].

Many honest OpenID Providers have historically produced non-canonical JWTs or JWTs with ambiguous claims due

to library bugs, non-compliant encoders, or implementation choices (as documented in prior works [18, 62, 68, 85, 86, 89, 94, 105, 107]). Moreover, malformed claims in JWTs are known to have security consequences beyond syntactic inconsistency. As observed in prior analyses of JWT implementations [62, 94], claims are routinely used by RPs to trigger lookups, database queries, or network requests (e.g., via `iss`, `sub`, `aud`, or `jku` URLs). In such cases, incorrectly validated or non-canonical claims have led to practical attacks including server-side request forgery (SSRF), directory traversal, and injection attacks¹². In a traditional OIDC context, the impact of these exploits is bounded by the short lifetime and scoped-use nature of the JWTs. In contrast, when the same malformed JWT is transformed into a reusable ZKA the system substantially magnifies the risk: an adversarial claim that would normally be discarded after one authentication event instead becomes a persistent authorization credential that may be reused indefinitely by any party.

These vulnerabilities reveal a deeper structural issue with the system: the correctness of zkLogin depends not only on JSON or JWT compliance, but on a *particular, unstated grammar* implicitly assumed by its proving circuits and parsers. The public documentation currently provides no canonical grammar for their JWT parsing, nor any constraint on whitespace handling, Unicode escapes, duplicate fields, claim ordering, or key uniqueness.

Acknowledged by zkLogin’s design. The zkLogin paper itself [8] highlights that allowing escape sequences inside JSON keys can lead to a break of security. In particular, it observes that if JSON keys contain escaped quotes, then security can potentially no longer hold (cf. Listing 3 in the zkLogin paper). This confirms that key canonicalization and unique-key enforcement are not merely interoperability concerns, but security-critical requirements for zkLogin-style claim binding. However, the paper treats escaped quotes primarily as an isolated corner case, and does not discuss the broader class of *character smuggling* and non-canonical encodings in claim key/values. Nor does it consider the downstream risk of *tainted claim strings*: even when the JWT signature is valid, claim values may contain attacker-controlled content (including JavaScript fragments) that later propagates into other parts of the system. Consequently, preventing claim-confusion requires not only ruling out the specific “escaped quote” pattern, but enforcing strict canonical parsing and sanitization across the full zkLogin proving and consumption pipeline.

Mitigation. A robust implementation of zkLogin must enforce *canonical JWT semantics* at the RP, salt and proving interface. At minimum, said services (and any reference implementation) should: (i) reject payloads whose decoded JSON contains duplicate keys, (ii) use a standards-compliant JSON

parser and prohibit non-canonical key encodings that may admit multiple parses, (iii) enforce well-typed claims for all security-relevant fields, and (iv) require and validate temporal OIDC claims (e.g., `exp`, `iat`) under the intended issuer policy. Crucially, these checks should not remain purely off-circuit: the zkLogin proof should be bound to a *canonicalized* interpretation of the JWT payload. In particular, the statement proven in zero knowledge should include that the prover has parsed the header and payload according to a unique, standards-compliant JSON grammar (including unique-key enforcement), and that the extracted security-critical claims (e.g., `iss`, `sub`, `aud`, `exp`, `iat`) satisfy the required OIDC validation rules. Otherwise, correctness continues to rely on external parsing conventions and implementation-specific claim extraction, reintroducing parser differentials. Additionally, downstream components that log or render claim values should treat them as untrusted input and apply appropriate sanitization. If these checks are delegated to RPs, then zkLogin security becomes dependent on application-specific parsing behavior, which is undesirable in federated deployments.

4.0.2 Cross-Impersonation via Missing Binding

In practice, as we have seen, the security of zkLogin rests on three key environmental assumptions: (i) external services maintain a correct issuer allow-list, and issuers reliably publish their signing keys at stable endpoints consistent with advertised `kid`; (ii) the `aud` value serves as an effective identifier for the RP and is correctly enforced when authorizing access to proving and salt services—despite being public and not cryptographically bound to the RP; and (iii) JWTs are well-formed and admit a unique, unambiguous interpretation under the (non-standard) parsing grammar implicitly assumed by the proving circuits. We show that each of these assumptions can be subverted in practice, enabling unauthorized invocation of proving and salt services, long-term misuse of JWT-derived authorization artifacts, and, eventually, cross-RP and cross-subject impersonation. Our goal in this subsection is to demonstrate end-to-end cross impersonation enabled by these assumptions. Concretely, we show that: (i) the issuer trust policy admits attacker-controlled issuers in common deployments; (ii) access control at the proving and salt services can be bypassed or misused due to missing binding between the requesting RP, subject and the JWT claims being proven; and (iii) the recommended browser-based deployment model exposes long-lived bearer artifacts and incorrectly relies on browser isolation as a security boundary.

On malicious issuers. The reference implementation of zkLogin¹³ extends the notion of a “trusted issuer” beyond an explicitly enumerated allow-list. While the documentation

¹²See, e.g., Section 2.9 (“Indirect Attacks on the Server”) of [94] or [66].

¹³See: https://web.archive.org/web/20251130200859/https://github.com/MystenLabs/fastcrypto/blob/main/fastcrypto-zkp/src/bn254/zk_login.rs

suggests that issuers are chosen from a fixed set of well-known providers, the official implementation dynamically accepts *any* Amazon Cognito user pool as a valid OIDC issuer. This behaviour is triggered purely by the syntactic form of the `iss` claim: instead of matching `iss` against a fixed set of trusted issuer URLs, the verifier interprets any issuer string matching an AWS Cognito URL template as automatically trusted.

Concretely, if the issuer string has the form `https://cognito-idp.region.amazonaws.com/tenant_id`, then the implementation treats it as a valid issuer. The `<region>` and `<tenant_id>`¹⁴ components are extracted directly from `iss` and used to derive both the issuer base URL and the endpoint from which keys are fetched. This means, that anyone can set up a malicious issuer at AWS Cognito to be used by zkLogin.

Although the hosted prover operated by Sui currently enforces its own issuer allow-list, this is a deployment choice rather than a property guaranteed by the zkLogin protocol or its public documentation. The particular instantiation of zkLogin in the Slush wallet does restrict the AWS issuers, but this is a deployment choice, not a protocol guarantee: it is further not authenticated by the RP, so it remains a static-list at the prover side which is, by documentation, considered untrusted. In particular, any third-party prover that follows the reference implementation without an additional out-of-band allow-list will, by default, accept *any* Cognito user pool as a trusted issuer. The resulting risk is therefore structural: it arises from issuer recognition by pattern matching and under-specified trust policy, rather than being confined to a specific prover instance. The zkLogin paper does acknowledge the fact that a malicious OP breaks unforgeability and unlikability (as the `salt` usage cannot be enforced); however, this is difficult to justify in federated OIDC settings where attacker-controlled issuers are feasible unless strict allow-listing and trust establishment are enforced at the protocol level.

More fundamentally, zkLogin treats issuer trust policy (the issuer allow-list) as a prover-dependent configuration choice, rather than as a first-class part of the protocol specification and overall verification semantics. This runs counter to well-established security practices: trust roots must be explicit, consistently enforced, and subject to auditing mechanisms. A useful analogy arises in the WebPKI ecosystem underpinning TLS deployments. While certificates are public objects, they are accepted only if they chain to a trusted root in a well-defined root store, and are increasingly monitored for misbehaviour through ecosystem-wide mechanisms such as Certificate Transparency [69]. Crucially, these trust decisions are not left to *ad-hoc* application decisions, but are treated as systemic requirements. Similarly, zkLogin should define and enforce issuer trust semantics at the protocol level (and bind them into proof verification where possible), rather than delegating them to mutable local configurations. We further note

that external services (including the prover) have no access to configuration changes of issuers (revoked `aud` or `sub`), so an allow-list needs to be enforced by all parties of the system, not only by some external services.

On unauthorized prover/salt service access. Although not explicitly documented in any public zkLogin materials, access to both the proving and salt services is frequently gated by an RP-specific API key registered with the service provider. In practice, this mechanism is used as an auxiliary authentication step when invoking the services. When examining the official wallet that integrates zkLogin [4], however, we observed that this API key is transmitted directly in standard HTTPS request headers from the browser environment, where it is observable by any local principal with code-execution capabilities (e.g., browser extensions, injected scripts, or debugging tooling). Because this API key uniquely identifies an RP, its disclosure enables an attacker to impersonate that RP when interacting with the external services. While this may be dismissed as a deployment misconfiguration, the vulnerability is structural: the zkLogin documentation provides no guidance for securely handling such credentials, and the API key is not cryptographically derived from protocol-bound values (e.g., `aud` or `iss`). Instead, it functions as a static bearer credential external to the trust relationships expressed by the zkLogin flow. As a result, the integrity of the proving and salt services depends not only on issuer or audience checks (public values), but also on the secrecy of a browser-exposed long-lived credential, whose compromise directly enables cross-RP impersonation.

Moreover, because the API key is not cryptographically bound to the `aud` value (or to any protocol-derived value), which serves as the public RP identifier given by `IS`, possession of the key does not constrain how it may be used. In particular, any party holding the RP's API key may request proofs for JWTs whose `aud` does not correspond to that RP and `IS`. In particular, an RP holding a valid API key may request proofs for JWTs issued under any of its registered `aud` values (as an RP can have many `aud` values per `IS` and different across `IS`), independently of the application, environment, or authorization context for which the JWT was originally intended. Maintaining a static allow-list of registered public `aud` values per RP does not prevent this behavior, as it enforces only set membership rather than binding proof generation to the specific `aud` and issuance context under which the JWT was obtained.

As a consequence, the proving service behaves as a generic proof-generation oracle for arbitrary JWTs, authorized solely by possession of a bearer API key. This enables an adversary who obtains an API key to generate proofs for identities and applications unrelated to the original RP and `aud`, while presenting those proofs as originating from a legitimate trusted RP and `aud` interaction. Such behaviour contradicts the intended OIDC semantics, in which `aud` binds a signed JWT to a specific RP and `iss`. More generally, the current design

¹⁴Both values are public identifiers.

admits proof generation without any authenticated binding between (i) the JWT’s RP identifier `aud` and (ii) the RP identity represented by the API key. Since the external services have no mechanism to validate this relationship against the **IS**, possession of a leaked API key suffices to mount cross-RP impersonation attacks. We confirmed this behaviour with surveys of the services.

This missing binding extends beyond the RP’s identifier `aud` to the subject identifier `sub` as well: `sub` may correspond to an email address (e.g., `alice@gmail.com`) or another issuer-defined identifier and is assigned by the identity provider **IS** on a per-user basis. In OIDC, `sub` is defined only relative to a particular issuer: it uniquely identifies a user within the namespace of **IS** and has no meaning independent of `iss`. Indeed, the OIDC specification [92] requires that a JWT be issued only if the user identified by `sub` has an active session with the issuer **IS**. Consequently, any interpretation of `sub` that is not bound to `iss` and to the issuance context is ill-defined. Since `sub` values are issuer-defined and unbounded, it is infeasible for the prover to register or pre-authorize subject identifiers. As a result, possession of an RP’s API key enables the RP to request proofs for JWTs corresponding to arbitrary subjects, without any mechanism at the proving interface to enforce that the subject identity used in the proof corresponds to the intended authorization context. Moreover, none of the values submitted to the external services establish that the requesting party is the legitimate holder of the JWT or that it controls the corresponding signing key `skU`.

A correct design of systems like zkLogin should enforce *RP-issuer-subject binding*, whereby proof generation is authorized only for a consistent tuple (`iss`, `sub`, `aud`, RP identity). This, however, might not be enough. Consider a malicious issuer that observes a public RP’s `aud` already registered at the prover service. **IS** then assigns this identifier (as a string) to a different RP who, in turn, registers the resulting (`iss`, `aud`) pair at the prover service. As a consequence, **IS** can legitimately issue syntactically valid JWTs containing its own `iss`, the reused `aud`, an attacker-chosen `sub`, and a signature generated using a key under its control, which are accepted by the prover service despite not corresponding to the original RP associated with that `aud`. Validation still succeeds: the verification key is retrieved from the hard-coded key set, the `iss` and `aud` fields match accepted values, and the `sub` is never checked or bound to the RP. In particular, the proving and salt services should ensure that the RP identity requesting a proof (as represented by the API key) is authorized to obtain proofs only for JWTs whose (`iss`, `aud`) corresponds to that RP, that `aud` are unique, and that the attested `sub` is interpreted under the corresponding issuer namespace. Concretely, the proving and salt services should enforce that:

1. the `aud` in the JWT corresponds to the RP that registered the API key *and was issued by IS*;

2. the `aud` and `sub` are unique;
3. the `sub` is valid under the namespace of the `iss`;
4. the ZKP attests not only to the presence and value of `iss`, `aud`, and `sub`, but to their binding to the authenticated RP identity represented by the API key;
5. no proof is generated for any combination of `iss`, `aud`, `sub` or API key that does not satisfy the binding constraints.

Because zkLogin provides neither a protocol-level mechanism nor guidance for enforcing these relationships, the correctness of the authentication-to-authorization boundary is implicitly delegated to application developers. The system thereby relies on static bearer tokens that are exposed in browser environments, and yet assumes that they will be used only in ways that respect issuer semantics and RP identity constraints. As a result, the current design enables cross-RP, and cross-subject misuse of JWT-derived proofs, despite superficially holding valid signatures from allowed **IS**.

We further validated these observations by reverse-engineering the Docker images [58] of the proving service recommended by the official documentation [32], noting that the proving service may be deployed by any organization. The image exposes multiple binaries, including the proving functionality implemented via RapidSnark [57]. While the image includes logic to validate an allow-list of accepted issuers and their corresponding registered `aud` values per RP, issuer support is fixed in code, whereas the audience allow-list is injected externally at deployment time, and the documentation provides no guidance to developers on how these lists should be constructed or maintained. Crucially, the presence of issuer and `aud` allow-lists does not prevent the impersonation vectors discussed above. Subject-level impersonation remains possible, as `sub` values are neither registered nor bound to the API key or issuance context. Moreover, misuse remains possible within the set of registered `aud` values per issuer, as the proving interface does not enforce any binding between a proof request and a specific authorization context associated with a given `aud`.

On browser trust assumptions and client-side credential exposure. In the vast majority of public documentation of zkLogin, it is assumed that private or authorization-critical values can be safely stored or transmitted within the browser environment. In fact, in the wallets we examined, sensitive values (including API keys, salts, and cryptographic material) were stored in browser-accessible state (e.g., `localStorage`, `sessionStorage`, or client-side JavaScript), and in some cases were transmitted directly from the front-end to the proving service—a practice explicitly discouraged in OAuth and OIDC deployments [61]. Because these values function as long-lived epoch-bounded authorization credentials rather

than scope-limited JWTs, exposing them to the browser enables passive extraction by any principal with code execution in the client context.

A secure design should avoid placing long-lived authorization credentials in the browser at all. Instead, such values should be managed by a backend component acting as the RP’s authenticated agent: the backend should hold the API key, retrieve or store the salt, and invoke the proving and salt services. The client should receive only short-lived, action-limited outputs, and never static credentials sufficient to generate new proofs. Absent such guidance and enforcement, zkLogin encourages browser-based applications to behave as if they were secure server-side principals, despite the lack of isolation guarantees appropriate for long-term authorization.

The zkLogin documentation *justifies* these design choices by appealing to JWT semantics and to the browser’s model. It argues that storing tokens and invoking the prover directly from the browser is safe because the JWT’s `aud` value “scopes the token to the client ID to prevent phishing attacks”¹⁵ and because “the same-origin policy for the proof prevents the JWT obtained for a malicious application from being used for zkLogin”¹⁶. Implicitly, the documentation assumes that: (i) the `aud` claim is reliably enforced by the surrounding identity infrastructure and therefore binds a token to a specific RP; and (ii) the browser origin model provides sufficient confidentiality and integrity to protect JWTs and to constrain which applications may transmit them to the proving service. Browser security mechanisms such as the Same Origin Policy (SOP) and Content Security Policy (CSP) do not provide the confidentiality or integrity guarantees implicitly assumed by zkLogin [49, 81, 96]. SOP isolates origins from one another, yet treats all same-origin scripts—including third-party libraries, analytics tags, advertisements, and injected dependencies—as fully trusted principals with access to that origin’s DOM and storage. CSP constrains which scripts may be loaded, but does not restrict the privileges of those scripts once executed.

Our analysis demonstrates that neither claim holds in practice. First, the proving and salt services do not enforce that `aud` corresponds to the RP `IS` associated with the API key, nor that the `aud` value was ever validly issued by that `IS`. Second, because the API key and salts are transmitted from and stored directly in browser environments, arbitrary scripts or debugging tools can simply reuse them to generate new proofs, rendering “same-origin protection” irrelevant. The origin model governs where requests can be sent from, not who they are sent *as*: a static bearer credential such as an API key or reusable salt bypasses origin-based isolation entirely. Thus, instead of preventing phishing or unauthorized proof generation, the current design enables it: tokens issued for one RP can be combined with stolen credentials belonging to an-

other, and proofs can be generated from any browser context with access to these bearer artifacts. The reliance on browser trust assumptions therefore amplifies, rather than mitigates, cross-RP impersonation.

End-to-end vulnerability. Overall, our findings show that an attacker who merely follows the guidance provided by the zkLogin documentation and paper, official demo, and reference code can fully subvert the intended trust model without exploiting any cryptographic weakness. By (i) registering an arbitrary AWS Cognito tenant as `IS`, (ii) extracting a RP’s API key from the browser and using a public `aud`, (iii) constructing a JWT containing that `aud` and a `sub` under the attacker-controlled issuer, and (iv) signing it with the Cognito tenant’s key and submitting it to the proving service as trusted RP, the attacker can obtain a valid zkLogin proof for another `aud` and `sub`. Nothing in the protocol specification or the documentation prevents such a proof from being generated or subsequently used by an attacker pretending to be the entity that the `sub` specifies. The vulnerability does not depend on protocol deviations or cryptanalysis: it arises solely from faithfully applying the documented integration steps, combined with the absence of binding between the RP identity, `IS` and the claims being attested in ZKP.

More generally, these issues are exacerbated by the use of JWT-derived proofs as reusable authorization credentials. In contrast, JWTs were originally designed as short-lived authentication artifacts: their self-contained nature makes revocation difficult, and any party in possession of the token may continue to use it until expiry [94]. Accordingly, prior work has repeatedly cautioned against using JWTs for long-lived session management or delegated authorization, particularly when tokens are stored or transmitted in environments lacking strong isolation guarantees [42]. When client-side storage or third-party submission is used, JWTs become readily accessible to injected scripts, browser extensions, or compromised dependencies [94], thereby amplifying the risk of token replay and cross-RP impersonation.

Validation and attack path. We demonstrate an end-to-end *attack path* for cross-RP impersonation under the stated assumptions. We validated (a) malformed-JWT acceptance and (b) API-key-based authorization behavior on public endpoints: the remaining steps follow from the reference code and documented issuer trust logic. Concretely, using our own test accounts and OIDC flows, we confirmed that the official proving service accepts malformed JWTs that would be rejected by standards-compliant validation, and we observed that RP API keys extracted from browser-based integrations are accepted for service access. For ethical reasons and to avoid generating impersonation-capable credentials, we did not deploy an attacker-controlled issuer or attempt to impersonate real users.

¹⁵Note, however, that the `aud` claim is frequently misused or inconsistently validated in practice [19].

¹⁶Paraphrased from the Sui Foundation documentation.

Mitigation. Mitigating the vulnerabilities presented requires elevating issuer trust and RP binding from deployment choices to protocol-level properties. At minimum, a secure zkLogin design should enforce:

1. **Issuer trust policy must be explicit and verifiable.** The set of trusted issuers must be explicitly specified and consistently enforced by both the external services and the verifier. Pattern-based issuer acceptance (trusting any AWS Cognito tenant) should be prohibited by default. More generally, issuer allow-lists should be treated as trust roots and bound to verification semantics rather than left as local configurations.

2. **Service authorization must be bound to protocol claims.** Access to external services must not rely on browser-exposed static bearer API keys. If such keys are used operationally, they must be cryptographically bound to the RP identity and to the intended claim tuple, (*iss*, *aud*) (where *aud* can be a list to avoid shadowing), and requests should be rejected unless this binding holds. In particular, proof generation should be authorized only for consistent tuples (*iss*, *aud*, *sub*, RP identity).

3. **RP-issuer-subject binding must be enforced.** The system must treat (*iss*, *sub*) as the principal identifier. Accordingly, proof generation and verification should enforce that *sub* is interpreted under the namespace of the corresponding *iss* and that the requesting RP is authorized for the corresponding *aud* under that issuer.

4. **Avoid browser-side long-lived secrets.** Authorization-critical values should be managed by a backend component acting as the RP’s authenticated agent. The client should not store static credentials in browser-accessible state, and should receive only short-lived, action-limited outputs. Browser policies such as SOP are insufficient for this.

4.0.3 Centralization and Privacy Risks

Although motivated by *decentralized* identity and privacy-preserving authentication, zkLogin effectively recentralizes trust around a small set of actors: a fixed set of issuers, out-sourced proving and salt services, and browser-based client environments that store long-lived bearer artifacts. As such, zkLogin is centralized even when working in a federated-friendly environment. It relies on a fixed set of issuers, and the validity of proofs ultimately depends on these issuers persisting their public keys, namespaces, and subject semantics for a period of time. Further, the proving and salt services maintain global state (including JWT storage, issuer-to-key mappings, and allow-lists) which must be trusted and updated. While the system might look as a “self-sovereign” one, in practice, it inherits governance assumptions of traditional identity systems.

Further, zkLogin centralizes not only on issuers, but also on user accounts belonging to those issuers: a user must possess

an identity anchored to them. Identities become action-usable only if mediated by web account systems, binding participation to pre-existing platform accounts. Rather than removing dependence on centralized identity providers, zkLogin embeds them into authorization infrastructure. Moreover, since each authentication flow is issuer-mediated, the **IS** can observe the RP that the user logs into on each use (via *aud* and redirect metadata), a well-known privacy limitation of federated SSO [37]. In addition, the issuer identity (*iss* and even *kid*) is revealed in the clear to system participants, which may facilitate cross-RP linkability and tracking when combined with other forwarded material.

By forwarding JWTs to external proving or salt services, zkLogin further exposes sensitive identity attributes (including profile information such as photos, email addresses, and other private data) to third parties that played no role in the user’s OIDC consent decision [41]. Notably, such attributes are frequently embedded directly in JWTs and are therefore disclosed whenever the token is transmitted for proof generation. The initial consent flow authorizes disclosure to the RP, *not* to an external prover operated by a different entity. Nothing in the OIDC flow informs the user that sensitive attributes will be transmitted to a third party for computations, nor that JWTs may persist there. This renders the **IS** consent semantically invalid: the user never consents to a secondary disclosure of their identity data.

Relatedly, the storage and retention of these tokens at external services is opaque. The zkLogin documentation provides no guarantees regarding how long JWTs are stored by the prover or salt service, whether they are persisted, inspected, or aggregated, or whether derived salts or key material are linkable across applications from their perspective. Users are not informed that their identifiers may be retained by a centralized proving service that serves multiple independent wallets and applications. Neither the official website nor the public integration documentation provides clear disclosure of these retention practices or of the resulting privacy implications. In particular, we found little explicit analysis of privacy and linkability risks arising from centralized proof generation, including whether a prover can correlate repeated uses across different RPs.

Moreover, the system does not clearly improve usability or user autonomy. It still requires both applications and users to retain long-lived secret values in order to function correctly: *salt* must be persistently stored and never lost; the API key used to request proofs must be preserved and protected by the RP; and ephemeral keys and nonce material must remain recoverable. Thus, the burden of secret management is not eliminated, but largely shifted from cryptographic wallets to opaque client-side storage and static bearer artifacts.

Mitigation. Addressing the above requires making data flows and trust relationships explicit, and minimizing the disclosure of identity attributes beyond what the user consents to

in the OIDC flow. Importantly, the proving interface should avoid receiving full JWTs whenever possible. Instead, proof generation should be designed to operate on minimal claim material (e.g., only those fields required for statement construction), and should exclude auxiliary profile claims not required for authorization, or it should be generated client-side.

Further, proving and salt services should provide public, auditable disclosures regarding token handling: what data is received (JWTs or extracted claims), whether it is stored, retention duration, access controls, and whether any aggregation/telemetry is performed. In addition, documentation should explicitly describe the resulting privacy and linkability implications for users.

4.1 Ethical Considerations

We followed a responsible disclosure process throughout this research. All findings were communicated to the zkLogin designers and Sui on 29 November 2025, and they acknowledged receipt on the same day. We received no further communication, and we followed-up with an email on the 4th of February, 2026. We received a reply on 5th February 2026. We note that the official documentation from MystenLabs/-Sui (the private company that originally designed and developed the Sui blockchain, Slush wallet and zkLogin) notes that zkLogin is outside of scope for their bug-bounty program [53]. Our disclosure included a detailed technical report covering all architectural, parsing, policy, and centralization vulnerabilities identified in this work. In particular, we explicitly recommended that:

1. strong, specification-compliant JWT parsing and validation be *enforced at the RP side*, rather than deferred to the proving service;
2. the system explicitly asks for the creation of an allow-list of issuers, and disallows patterns as the AWS one;
3. a cryptographic binding be introduced between the issuer, subject, audience, and API key (e.g., using a commitment to all of them for each service, issuer and subject registered at the proving and salt service), preventing arbitrary proof generation. Furthermore, that a check be enforced to the RP to show the `aud` was assigned by the OP and it is the current one (not rotated);
4. all sensitive long-lived authorization material (salts, API keys, ephemeral keys) be prohibited from storage in browser-accessible environments, or front-end JavaScript variables;
5. the API key never be exposed to the browser front-end, and that wallets be updated to remove this pattern;

6. the proving and salt services adopt stricter verification rules, including canonical JSON enforcement, uniqueness checks, rejection of duplicate or malformed claims, and validation of key identifiers.
7. wallets obtain explicit user consent before forwarding any sensitive JWT to an external proving service, in line with OIDC privacy expectations and informed consent.

In their response, Sui addressed only points (4) and (5), focusing on the claim that exposing the API key and related parameters to the browser is acceptable because the RP must invoke the external services, and requests are “bound” to the wallet’s JWT, with all communication protected by TLS. Our findings indicate that these conditions do not provide a robust security boundary. First, the API key is a bearer credential: possession suffices to invoke the external services from arbitrary contexts contradicting any implicit origin- or RP-binding guarantee. Second, TLS is orthogonal to the relevant attacker models: injected scripts, malicious extensions, compromised dependencies, or brief device access observe decrypted data post-TLS and can trivially exfiltrate and replay JS-accessible bearer artifacts. This data also lives longer than OIDC sessions and can be used at any time by an attacker that saw the credentials once. Third, the current design offers poor incident containment: a static browser-exposed key prevents targeted revocation, whereas standard web patterns (short-lived session credentials, HttpOnly cookies, or proof-of-possession bindings), proxy-based services or OAuth’s Authorization Code Flow with Proof Key for Code Exchange (PKCE) [7] preserve security while reducing replay and exfiltration impact. We followed up on February 6, 2026, recommending that, if the workflow must remain client-driven, the design adopt these standard alternatives.

Following private disclosure in November 2025, we received a public response on February 14, 2026 via X (formerly Twitter) [52]. The response clarified that the Sui deployment of zkLogin enforces an allow-list of identity providers at the implementation level. Our analysis, however, concerns protocol-level guarantees and the security properties of zkLogin as a reusable primitive: we argue that such checks should be enforced as part of the protocol specification rather than relying on deployment-specific configuration. We have clarified these points to avoid misinterpretation. The response further stated that cross-issuer impersonation is not possible. Our paper does not claim cross-issuer impersonation; rather, we analyze cross-RP and cross-subject impersonation within a single issuer namespace. We have clarified these points to avoid misinterpretation. The response also makes explicit that the considered threat model assumes all issuers to be trusted. While we consider this assumption unrealistic in many deployment settings, particularly given the nature of OIDC and that many of the JWT values are public, the response indicates plans to integrate mitigations addressing malicious or misconfigured issuers, including adoption of fully standards-

compliant JWT/JSON parsing. At the time of writing, we are not aware of whether additional recommended mitigations have been implemented, nor of a timeline for the integration of standards-compliant parsing. We explicitly communicated our willingness to provide technical guidance on remediation.

We emphasize that at no point did we attempt to exploit these issues in production systems. All experimental results were obtained using our own test accounts, locally generated JWTs, and public testnet proving endpoints, and we never attempted to access, extract, or interact with any real user data. We did not attempt to gain elevated privileges or disrupt system functionality. Our reverse-engineering efforts were limited to inspection of publicly distributed Docker images and binaries that are explicitly intended to be deployed by third parties. We did not bypass technical protection measures, authentication controls, or licensing restrictions, nor did we modify or redistribute the analyzed artifacts. Reverse-engineering was performed solely to understand documented security-relevant behavior that is not specified in the public documentation and was necessary to assess the real-world security properties of the system.

We conducted this research strictly within ethical guidelines for security analysis, avoided generating harmful proofs or forging identities belonging to others, and limited all interactions to minimal inputs required to demonstrate the feasibility of the vulnerabilities described. We will continue to cooperate fully with the affected developers to ensure fixes are deployed responsibly and users are protected.

5 Discussion

Our analysis demonstrates that the foundational security assumptions underlying zkLogin cannot hold under realistic adversarial conditions. Together, the vulnerabilities presented undermine the core guarantees it claims to provide: scoped authorization, input integrity, unlinkability, and decentralization. Rather than reflecting isolated implementation bugs, these vulnerabilities arise from the architectural design choices that structurally re-purpose short-lived web authentication tokens into long-lived authorization credentials. A common theme across our failures is the way they compose all-together. Parsing ambiguities allow an attacker to craft adversarially shaped JWTs: this interacts directly with the absence of binding validation, enabling malformed tokens and audience confusion. These issues get compounded with trust amplification at external proving and salt services, where JWTs and identity attributes are forwarded and reused outside the scope of the **IS** and user consent. Finally, the exposure of cryptographic values in the browser turns these logical flaws into practical exploits: any malicious script or injected dependency can harvest them and generate fresh proofs indefinitely.

The vulnerabilities we document are distributed across the entire zkLogin pipeline. At the identity layer, the system does not enforce correct issuer–audience–subject bindings,

nor does it require strict temporal validity or canonical token structure. At the proving layer, the circuits validate only narrow byte-slices of a token and rely on assumptions fundamentally incompatible with JSON or OIDC semantics. At the verifier layer, validators rely on incomplete or stale issuer metadata, and no component checks whether the JWT’s intended scope aligns with the RP requesting authorization. Each subsystem independently weakens the intended security model: in aggregate, they negate it.

These outcomes highlight a deeper issue: systems extending web authentication tokens into cryptographic authorization mechanisms lack a unified and principled security framework. Instead, the design of zkLogin appears to have evolved by layering isolated components (OIDC semantics, *ad hoc* parsing logic, outsourced proving services, and blockchain expiry rules) without a global threat model governing how these components interact. As with other federated identity systems, the absence of a formal end-to-end analysis leads to brittle security boundaries, unexpected data flows, and unrevocable trust relationships that outlive the very tokens from which they derive.

We stress that these risks are magnified when systems are such are proposed for higher-stakes identity settings, such as digital passports, national ID systems, or government-backed attestations [38, 111, 112]. Extending web-token semantics to these domains, while simultaneously weakening issuer, audience, and consent guarantees, risks entrenching centralized identity providers as long-term authorization oracles, exposing sensitive personal attributes to external provers, and removing meaningful revocation or policy control. Such systems should not be proposed and deployed without a rigorous, formal treatment of their security and privacy properties, and without reconsidering whether repurposing web authentication tokens is appropriate for these application domains at all.

References

- [1] 1News. Debate rages as australia set to ban children from social media. <https://tinyurl.com/3vzhnmtc>, September 2024. Archived via Wayback Machine; Accessed July 2025.
- [2] New America. Exploring privacy-preserving age verification: A close look at zero-knowledge proofs. <https://www.newamerica.org/oti/briefs/exploring-privacy-preserving-age-verification/>, July 2025.
- [3] Sebastian Angel, Andrew J. Blumberg, Eleftherios Ioannidis, and Jess Woods. Efficient representation of numerical optimization problems for SNARKs. 2022.
- [4] Slush App. Slush wallet, 2025. Accessed: 2025-11-26. URL: <https://slush.app/>.
- [5] Suiet App. Suiet wallet, 2025. Accessed: 2025-11-26. URL: <https://suiet.app/>.
- [6] ARPA Canada. Age verification bill reintroduced in the senate. <https://arpacanada.ca/articles/age-verification-bill-reintroduced-in-the-senate/>, 2025. Accessed July 2025.
- [7] auth0 Docs. Authorization Code Flow with Proof Key for Code Exchange (PKCE). Accessed: 2026-01-25. URL: <https://auth0.com/docs/get-started/authentication-and-authorization-flow/authorization-code-flow-with-pkce>.
- [8] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindström, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. zklogin: Privacy-preserving blockchain authentication with existing credentials. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 3182–3196, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690356.
- [9] Foteini Baldimtsi and Deepak Maram. zkLogin: Privacy-Preserving Blockchain Auth (RWC 2025), 2025. Real World Crypto (RWC) 2025 talk, Accessed: 2025-11-29. URL: <https://www.youtube.com/watch?v=4PQ0JOLE1OI>.
- [10] T. Bray, D. Hollander, and N. A. Hall. The JavaScript Object Notation (JSON) Data Interchange Format. Technical Report RFC 8259, Internet Engineering Task Force (IETF), Dec 2017. Accessed: 2025-11-25. URL: <https://datatracker.ietf.org/doc/html/rfc8259>, doi:10.17487/RFC8259.
- [11] S. Celi, K. den Hartog, and H. Haddadi. Private and decentralized age verification architecture. Slides, IAB/W3C Workshop on Age-Based Restrictions on Content Access (agews), 2025. URL: <https://tinyurl.com/bdtvs9t9>.
- [12] Sofia Celi. The limits of zero-knowledge for age-verification. <https://brave.com/blog/zkp-age-verification-limits/>, November 2025. Brave Software blog, Accessed 2025-11-28.
- [13] Sofia Celi, Alex Davidson, Hamed Haddadi, Gonçalo Pestana, and Joe Rowell. DiStefano: Decentralized infrastructure for sharing trusted encrypted facts and nothing more. Cryptology ePrint Archive, Paper 2023/1063, 2023. URL: <https://eprint.iacr.org/2023/1063>.
- [14] Sylvain Chatel, Christian Knabenhans, Wouter Lueks, Mathilde Raynal, Carmela Troncoso, and Ádám Vécsei. Limitations and pitfalls of integrating pets in online age verification. Slides, IAB/W3C Workshop on Age-Based Restrictions on Content Access (agews), 2025. URL: <https://tinyurl.com/72erwdhj>.
- [15] OpenZeppelin community (spalladino). Sign in with Google to your Identity Contract (for fun and profit). OpenZeppelin Community Forum post, October 2019. Accessed 2025-11-28. URL: <https://tinyurl.com/hpn29xxx>.
- [16] Connect2id. Connect2id server, 2024. Accessed: 2025-11-25. URL: <https://connect2id.com/products/server>.
- [17] Cybersecurity and Infrastructure Security Agency (CISA). TraderTraitor: North Korean State-Sponsored APT Targets Blockchain Companies, 2022. Accessed: 2025-11-25. URL: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-108a>.
- [18] d0ge. SignSaboteur, 2025. Accessed: 2025-11-27. URL: <https://github.com/d0ge/sign-saboteur>.
- [19] Descope. How to use the JWT aud claim securely. Blog post, January 2025. Accessed: 2025-11-28. URL: <https://www.descope.com/blog/post/jwt-aud-claim>.
- [20] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983. doi:10.1109/TIT.1983.1056650.
- [21] doubleblind xyz. Double-blind, 2025. Accessed: 2025-11-25. URL: <https://github.com/doubleblind-xyz/double-blind>.

- [22] European Digital Rights (EDRi). Showing your id to get online might become a reality – a closer look at the EU’s new age verification app. <https://tinyurl.com/7ssexsan>, May 2025.
- [23] European Commission. 2030 Digital Compass: The European Way for the Digital Decade. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=celex:52021DC0118>, 2021. COM(2021) 118 final.
- [24] European Commission. Proposal for a Regulation of the European Parliament and of the Council amending Regulation (EU) No 910/2014 as regards establishing a framework for a European Digital Identity. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52021PC0281>, June 2021.
- [25] Zhiyong Fang, David Darais, Joseph P. Near, and Yupeng Zhang. Zero knowledge static program analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS ’21*, page 2951–2967, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3460120.3484795.
- [26] Daniel Fett, Ralf Küsters, and Guido Schmitz. A Comprehensive Formal Security Analysis of OAuth 2.0. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, CCS ’16*, page 1204–1215, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2976749.2978385.
- [27] OpenID Foundation. How openid connect works, 2025. Accessed: 2025-11-26. URL: <https://openid.net/developers/how-connect-works/>.
- [28] Sui Foundation. A Complete Guide to zkLogin: How it Works and How to Use It, 2023. YouTube video. Accessed: 2025-11-26. URL: <https://www.youtube.com/watch?v=Jk4mq5IOUYc>.
- [29] Sui Foundation. Configure OpenID Providers for zkLogin, 2025. Accessed: 2025-11-25. URL: <https://docs.sui.io/guides/developer/cryptography/zklogin-integration/developer-account>.
- [30] Sui Foundation. zkLogin, 2025. Accessed: 2025-11-25. URL: <https://docs.sui.io/concepts/cryptography/zklogin>.
- [31] Sui Foundation. zkLogin Example, 2025. Accessed: 2025-11-25. URL: <https://docs.sui.io/guides/developer/cryptography/zklogin-integration/zklogin-example>.
- [32] Sui Foundation. zkLogin Integration Guide, 2025. Accessed: 2025-11-25. URL: <https://docs.sui.io/guides/developer/cryptography/zklogin-integration>.
- [33] Matteo Frigo and abhi shelat. Anonymous credentials from ECDSA. Cryptology ePrint Archive, Paper 2024/2010, 2024. URL: <https://eprint.iacr.org/2024/2010>.
- [34] Kevin Fu, Emil Sit, Kendra Smith, and Nick Feamster. The dos and don’ts of client authentication on the web. In *10th USENIX Security Symposium (USENIX Security 01)*, Washington, D.C., August 2001. USENIX Association. URL: <https://www.usenix.org/conference/10th-usenix-security-symposium/dos-and-donts-client-authentication-web>.
- [35] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). pages 291–304, 1985. doi:10.1145/22145.22178.
- [36] Paul Grubbs, Arasu Arun, Ye Zhang, Joseph Bonneau, and Michael Walfish. Zero-Knowledge middleboxes. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 4255–4272, Boston, MA, August 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/grubbs>.
- [37] Sven Hammann, Ralf Sasse, and David Basin. Privacy-preserving openid connect. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, ASIA CCS ’20*, page 277–289, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3320269.3384724.
- [38] Alexis Hancock and Paige Collings. Zero knowledge proofs alone are not a digital id solution to protecting user privacy. <https://tinyurl.com/2kxz433k>, July 2025. Electronic Frontier Foundation blog, Accessed 2025-11-28.
- [39] Ethan Heilman, Lucie Mugnier, Athanasios Filippidis, Sharon Goldberg, Sebastien Lipman, Yuval Marcus, Mike Milano, Sidhartha Premkumar, Chad Unrein, and John Merfeld. OpenPubkey: Augmenting OpenID connect with user held signing keys. Cryptology ePrint Archive, Paper 2023/296, 2023. URL: <https://eprint.iacr.org/2023/296>.
- [40] iden3. Circom 2 Documentation, 2025. Accessed: 2025-11-25. URL: <https://docs.circom.io/>.

- [41] Jacob Ideskog. The relationship between consent and claims, Feb 2025. Accessed: 2025-11-28. URL: <https://curity.io/resources/learn/the-relationship-between-consent-and-claims/>.
- [42] Joepie. Stop Using JWT for Sessions. <http://crypto.net/~joepie91/blog/2016/06/13/stop-using-jwt-for-sessions/>, 2016.
- [43] Ash Johnson. The Path to Digital Identity in the United States. Report, Information Technology and Innovation Foundation, September 2024. Accessed July 2025. URL: <https://itif.org/publications/2024/09/23/path-to-digital-identity-in-the-united-states/>.
- [44] M. B. Jones, J. Bradley, and N. Sakimura. JSON Web Signature (JWS). Technical Report RFC 7515, Internet Engineering Task Force (IETF), May 2015. Accessed: 25 Nov. 2025. URL: <https://www.rfc-editor.org/rfc/rfc7515.txt>, doi:10.17487/RFC7515.
- [45] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). Request for Comments RFC 7519, Internet Engineering Task Force (IETF), May 2015. URL: <https://datatracker.ietf.org/doc/html/rfc7519>.
- [46] Jovi Cheng. Sui’s ZkLogin Demo, 2025. Accessed: 2025-11-25. URL: <https://github.com/jovicheng/sui-zklogin-demo>.
- [47] jovicheng. sui-zklogin-demo. GitHub repository, 2023. Accessed: 2025-11-28. URL: <https://github.com/jovicheng/sui-zklogin-demo>.
- [48] Jow Tidy. North Korean hackers cash out hundreds of millions from \$1.5bn ByBit hack, 2025. Accessed: 2025-11-25. URL: <https://www.bbc.com/news/articles/c2kgndw7l0>.
- [49] Gayatri Priyadarsini Kancherla, Dishank Goel, and Abhishek Bichhawat. Least privilege access for persistent storage mechanisms in web browsers. In *Proceedings of the ACM on Web Conference 2025*, WWW ’25, page 4832–4840, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3696410.3714887.
- [50] Kaitlyn Kenwell. Strict validation breaks due to Microsoft’s OIDC noncompliance, 2023. Accessed: 2025-11-25. URL: <https://github.com/ramosbugs/openidconnect-rs/issues/122>.
- [51] M. Kirova. Overview of Pre-notified and Notified eID Schemes under eIDAS. eID User Community – European Commission Digital Building Blocks, October 2023. Accessed 2025-09-19. URL: <https://ec.europa.eu/digital-buildingblocks/sites/spaces/EIDCOMMUNITY/pages/48762251/Overviewofpre-notifiedandnotifiedeID+schemesunder+eIDAS>.
- [52] Kostas Kryptos. Public response regarding zkLogin security analysis (X/Twitter), February 2026. Accessed: 2026-02-15. URL: <https://x.com/kostascrypto/status/2022475095722123322>.
- [53] Mysten Labs. Bug bounty program. <https://www.mystenlabs.com/bug-bounty>, 2023. Accessed 2025-11-30.
- [54] Mysten Labs. Enoki documentation, 2025. Accessed: 2025-11-26. URL: <https://docs.enoki.mystenlabs.com/>.
- [55] Mysten Labs. fastcrypto, 2025. Accessed: 2025-11-25. URL: <https://github.com/MystenLabs/fastcrypto>.
- [56] Mysten Labs. Sui: The Sui blockchain and related tooling, 2025. Accessed: 2025-11-26. URL: <https://github.com/MystenLabs/sui/tree/main>.
- [57] Mysten Labs. RapidSNARK, 2026. Accessed: 2026-01-25. URL: <https://github.com/MystenLabs/rapidsnark>.
- [58] Mysten Labs. zkLogin Docker Image, 2026. Official image, Accessed: 2026-01-25. URL: <https://hub.docker.com/r/mysten/zklogin>.
- [59] Lakshmiraghavan, Badrinarayanan. *Web Tokens*, pages 191–225. Apress, Berkeley, CA, 2013. doi:10.1007/978-1-4302-5783-7_10.
- [60] Jan Lauinger, Jens Ernstberger, Andreas Finkenzeller, and Sebastian Steinhorst. Janus: Fast privacy-preserving data provenance for TLS. 2025.
- [61] Torsten Lodderstedt, Mark McGloin, and Phil Hunt. OAuth 2.0 Threat Model and Security Considerations. Technical Report RFC 6819, Internet Engineering Task Force (IETF), January 2013. Accessed: 2025-11-27. URL: <https://datatracker.ietf.org/doc/html/rfc6819>.
- [62] PortSwigger Ltd. JWT attacks. <https://portswigger.net/web-security/jwt>, 2025. Web Security Academy – PortSwigger.
- [63] PortSwigger Ltd. OAuth. <https://portswigger.net/web-security/oauth>, 2025. Web Security Academy – PortSwigger.

- [64] Kaixuan Luo, Xianbo Wang, Pui Ho Adonis Fung, Wing Cheong Lau, and Julien Lecomte. Universal cross-app attacks: exploiting and Securing OAuth 2.0 in integration platforms. In *Proceedings of the 34th USENIX Conference on Security Symposium, SEC '25*, USA, 2025. USENIX Association.
- [65] Christian Mainka, Vladislav Mladenov, and Jörg Schwenk. Do Not Trust Me: Using Malicious IdPs for Analyzing and Attacking Single Sign-on. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 321–336, 2016. doi:10.1109/EuroSP.2016.33.
- [66] Christian Mainka, Vladislav Mladenov, Jörg Schwenk, and Tobias Wich. SoK: Single Sign-On Security - An Evaluation of OpenID Connect. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 251–266, 2017. doi:10.1109/EuroSP.2017.32.
- [67] Carlo Mazzocca, Abbas Acar, Selcuk Uluagac, Rebecca Montanari, Paolo Bellavista, and Mauro Conti. A survey on decentralized identifiers and verifiable credentials. *IEEE Communications Surveys & Tutorials*, pages 1–1, 2025. doi:10.1109/COMST.2025.3543197.
- [68] Tim McLean. Critical vulnerabilities in JSON Web Token libraries. <https://tinyurl.com/34zdxfy2>, August 2020. Guest Author post on the Auth0 Blog.
- [69] Sarah Meiklejohn, Joe DeBlasio, Devon O'Brien, Chris Thompson, Kevin Yeo, and Emily Stark. SoK: SCT Auditing in Certificate Transparency, 2022. URL: <https://arxiv.org/abs/2203.01661>, arXiv:2203.01661.
- [70] Vladislav Mladenov and Christian Mainka. Attacking OpenID Connect 1.0 - Malicious Endpoints Attack, 2015. Accessed: 2025-11-25. URL: <https://web-in-security.blogspot.com/2015/10/attacking-openid-connect-10-malicious.html>.
- [71] Vladislav Mladenov, Christian Mainka, and Jörg Schwenk. On the security of modern Single Sign-On Protocols: Second-Order Vulnerabilities in OpenID Connect, 2016. URL: <https://arxiv.org/abs/1508.04324>, arXiv:1508.04324.
- [72] MojoAuth. Implement OAuth 2.0 and OpenID Connect (OIDC) with Quarkus, December 2025. URL: <https://mojoauth.com/oauth2-oidc/implement-oauth2-oidc-with-quarkus>.
- [73] MystenLabs. ts-sdks: Sui TypeScript SDK - zklogin module, 2025. Accessed: 2025-11-27. URL: <https://github.com/MystenLabs/ts-sdks/tree/main/packages/typescript/src/zklogin>.
- [74] Neha Narula, Willy Vasquez, and Madars Virza. zk-Ledger: Privacy-Preserving auditing for distributed ledgers. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 65–80, Renton, WA, April 2018. USENIX Association. URL: <https://www.usenix.org/conference/nsdi18/presentation/narula>.
- [75] Asmit Nayak, Rishabh Khandelwal, Earlene Fernandes, and Kassem Fawaz. Experimental security analysis of sensitive data access by browser extensions. In *Proceedings of the ACM Web Conference 2024, WWW '24*, page 1283–1294, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3589334.3645683.
- [76] David Naylor, Richard Li, Christos Gkantsidis, Thomas Karagiannis, and Peter Steenkiste. And then there were more: Secure communication for more than two parties. In *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies, CoNEXT '17*, page 88–100, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3143361.3143383.
- [77] Aviad Hahami Unit42 Palo Alto Networks. OH-MY-DC: OIDC Misconfigurations in CI/CD, 2025. Accessed: 2025-11-25. URL: <https://unit42.paloaltonetworks.com/oidc-misconfigurations-in-ci-cd/>.
- [78] Shradha Neupane, Grant Holmes, Elizabeth Wyss, Drew Davidson, and Lorenzo De Carli. Beyond typosquatting: An in-depth look at package confusion. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3439–3456, Anaheim, CA, August 2023. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/neupane>.
- [79] AP News. Utah lawsuit over online porn age-verification dismissed. *Associated Press*, February 2025. Accessed July 2025. URL: <https://tinyurl.com/y24nnk78>.
- [80] Critical Security Oy. Targeted review of certain Sui Wallet updates. Technical report, Sui Foundation Security Audits, 2023. Accessed: 2025-11-25. URL: https://github.com/sui-foundation/security-audits/blob/main/docs/sui_wallet_zklogin_update_H223_review_v_0_9.pdf.

- [81] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. Zanzibar: Google’s consistent, global authorization system. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 33–46, Renton, WA, July 2019. USENIX Association. URL: <https://www.usenix.org/conference/atc19/presentation/pang>.
- [82] Sanghyeon Park, Jeonghyuk Lee, Seunghwa Lee, Jung Hyun Chun, Hyeonmyeong Cho, Mingi Kim, Hyun Ki Cho, and Soo-Mook Moon. Beyond the blockchain address: Zero-knowledge address abstraction. In *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing, SAC ’25*, page 366–374, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3672608.3707839.
- [83] Kenneth G. Paterson, Matteo Scarlata, and Kien Tuong Truong. Three lessons from threema: Analysis of a secure messenger. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1289–1306, Anaheim, CA, August 2023. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/paterson>.
- [84] Felipe Pezoa, Juan L. Reutter, Fernando Suarez, Martín Ugarte, and Domagoj Vrgoč. Foundations of JSON Schema. In *Proceedings of the 25th International Conference on World Wide Web, WWW ’16*, page 263–273, Republic and Canton of Geneva, CHE, 2016. International World Wide Web Conferences Steering Committee. doi:10.1145/2872427.2883029.
- [85] PortSwigger. JavaScript Object Signing and Encryption Pentesting Helper (JOSEPH), 2021. Accessed: 2025-11-27. URL: <https://github.com/PortSwigger/json-web-token-attacker>.
- [86] PortSwigger. JWT Editor, 2025. Accessed: 2025-11-27. URL: <https://github.com/PortSwigger/jwt-editor>.
- [87] Melissa Quinn. Supreme court upholds texas law on age verification for porn sites. *CBS News*, June 2025. Accessed July 2025. URL: <https://tinyurl.com/yrxhjk8t>.
- [88] Ed R. Hedberg, M.B. Jones, A.Å. Solberg, J. Bradley, G. De Marco, and V. Dzhuvinov. OpenID Federation 1.0 - draft 48, 2026. Accessed: 2026-01-15. URL: https://openid.net/specs/openid-federation-1_0.html.
- [89] Brendan Rius. JWT Cracker, 2025. Accessed: 2025-11-27. URL: <https://github.com/brendan-rius/c-jwt-cracker>.
- [90] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure. Cryptology ePrint Archive, Paper 2022/878, 2022. URL: <https://eprint.iacr.org/2022/878>.
- [91] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore. OpenID Connect Core 1.0. Technical Report Final, OpenID Foundation, Feb 2014. Accessed: 25 Nov. 2025. URL: https://openid.net/specs/openid-connect-core-1_0-final.html.
- [92] N. Sakimura, J. Bradley, M.B. Jones, B. de Medeiros, , and C. Mortimore. OpenID Connect Core 1.0 incorporating errata set 2, 2023. Accessed: 2025-11-25. URL: https://openid.net/specs/openid-connect-core-1_0.html.
- [93] Luca Schönwälder. A Security Analysis of OpenID Connect. Master’s thesis, Ruhr-Universität Bochum, May 2021. URL: <https://www.nds.ruhr-uni-bochum.de/media/ei/arbeiten/2021/05/03/masterthesis.pdf>.
- [94] Yaron Sheffer, Dick Hardt, and Michael B. Jones. JSON Web Token Best Current Practices. Request for Comments RFC 8725, Internet Engineering Task Force (IETF), February 2020. Best Current Practice, BCP 225. URL: <https://www.rfc-editor.org/rfc/rfc8725.html>.
- [95] Shinami. ZKLogin Wallet API — Sui Wallet Services Documentation. <https://docs.shinami.com/api-docs/sui/wallet-services/zklogin-wallet-api>, 2025. Accessed: 2025-11-28.
- [96] Dolière Francis Somé. Empoweb: Empowering web applications with browser extensions. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 227–245, 2019. doi:10.1109/SP.2019.00058.
- [97] Alan Stapelberg. Opening up Zero-Knowledge Proof technology to promote privacy in age assurance. Blog post, Google, July 2025. <https://tinyurl.com/34mmhpy9>, Accessed 2025-11-28.
- [98] Sui. Sui’s zklogin Demo Web App. <https://sui-zklogin.vercel.app/>, 2025. Accessed: 2025-11-25.
- [99] Sui Foundation. Sui official website. <https://sui.io/>, 2025. Accessed: 2025-11-28.

- [100] European Data Protection Supervisor. Techdispatch #3/2025 - digital identity wallets. <https://tinyurl.com/23m2ybm5>, December 2025.
- [101] Surftech. Surf wallet. <https://www.surf.tech/>, 2025. Accessed: 2025-11-28.
- [102] Christophe Tafani-Dereeper. Abusing Misconfigured OIDC Authentication In Cloud Environments, 2025. Accessed: 2025-11-25. URL: <https://www.youtube.com/watch?v=r68fyFhkeV0>.
- [103] Justin Martin (TheFrozenFire). snark-jwt-verify: Verify JWTs using SNARKs, 2024. Accessed: 2025-11-25. URL: <https://github.com/TheFrozenFire/snark-jwt-verify>.
- [104] Tinder. Identifying vulnerabilities in GitHub Actions & AWS OIDC Configurations, 2023. Accessed: 2025-11-25. URL: <https://tinyurl.com/5yyu7wyn>.
- [105] Abhishek Tiwari. JWTForge: A JWT Vending Service for Testing, Fuzzing, and Security Research of OAuth2/OIDC Implementations. *Front Matter*, November 2025. doi:10.59350/6pdm-3cm41.
- [106] TLSNotary Development Team. Tlsnotary. <https://tlsnotary.org/>, 2025. Accessed 2025-11-30.
- [107] Andy Tyler. jwttool: A toolkit for testing, tweaking and cracking JSON Web Tokens, 2025. GitHub repository. URL: https://github.com/ticarpi/jwt_tool.
- [108] UK Government. Digital identity. <https://www.gov.uk/guidance/digital-identity>, 2025. Accessed July 2025.
- [109] EU Digital Identity Wallet. G: Zero Knowledge Proof. <https://eu-digital-identity-wallet.github.io/eudi-doc-architecture-and-reference-framework/latest/discussion-topics/g-zero-knowledge-proof/>, 2024. Accessed July 2025.
- [110] WHATWG. HTML Living Standard — Web storage, 2025. Accessed: 2025-11-27. URL: <https://html.spec.whatwg.org/multipage/webstorage.html>.
- [111] Svea Windwehr and Alexis Hancock. Age verification in the european union: The commission’s age verification app. <https://www.eff.org/deeplinks/2025/04/age-verification-european-union-mini-id-wallet>, April 2025. Electronic Frontier Foundation blog - Accessed 2026-01-12.
- [112] Svea Windwehr and Alexis Hancock. Digital Identities and the Future of Age Verification in Europe. <https://tinyurl.com/2f37t6vs>, April 2025. Electronic Frontier Foundation blog - Accessed 2026-01-12.
- [113] Xiang Xie, Kang Yang, Xiao Wang, and Yu Yu. Lightweight authentication of web data via garble-then-prove. In *Proceedings of the 33rd USENIX Conference on Security Symposium*, SEC ’24, USA, 2024. USENIX Association.
- [114] Collin Zhang, Zachary DeStefano, Arasu Arun, Joseph Bonneau, Paul Grubbs, and Michael Walfish. Zombie: Middleboxes that Don’t snoop. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1917–1936, Santa Clara, CA, April 2024. USENIX Association. URL: <https://www.usenix.org/conference/nsdi24/presentation/zhang-collin>.
- [115] Fan Zhang, Ethan Cecchetti, Kyle Croman, Ari Juels, and Elaine Shi. Town crier: An authenticated data feed for smart contracts. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’16, page 270–282, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2976749.2978326.
- [116] Fan Zhang, Deepak Maram, Harjasleen Malvai, Steven Goldfeder, and Ari Juels. Deco: Liberating web data using decentralized oracles for tls. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, CCS ’20, page 1919–1938, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3372297.3417239.
- [117] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 995–1010, Santa Clara, CA, August 2019. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman>.
- [118] zkID team of the Ethereum Foundation. zkID: A Step Towards Privacy-Preserving Digital Identity. <https://github.com/privacy-ethereum/zkID/blob/main/paper/zkID.pdf>, 2025. Accessed 2025-11-28.
- [119] zkPass. zkPass: zkTLS. <https://docs.zkpass.org/overview/introduction>, 2026.
- [120] zkSecurity. Public Report of Sui’s zkLogin Audit, Nov 2023. Accessed: 2025-11-25. URL: <https://blog.zksecurity.xyz/posts/zklogin/>.

- [121] zkSecurity. Audit of Mysten Lab’s zkLogin circuits and ceremony. Technical report, Sui Foundation Security Audits, 2024. Accessed: 2025-11-25. URL: https://github.com/sui-foundation/security-audits/blob/main/docs/zksecurity_zklogin-circuits.pdf.